

数据与算法 - 第5讲

树、二叉树 (一)

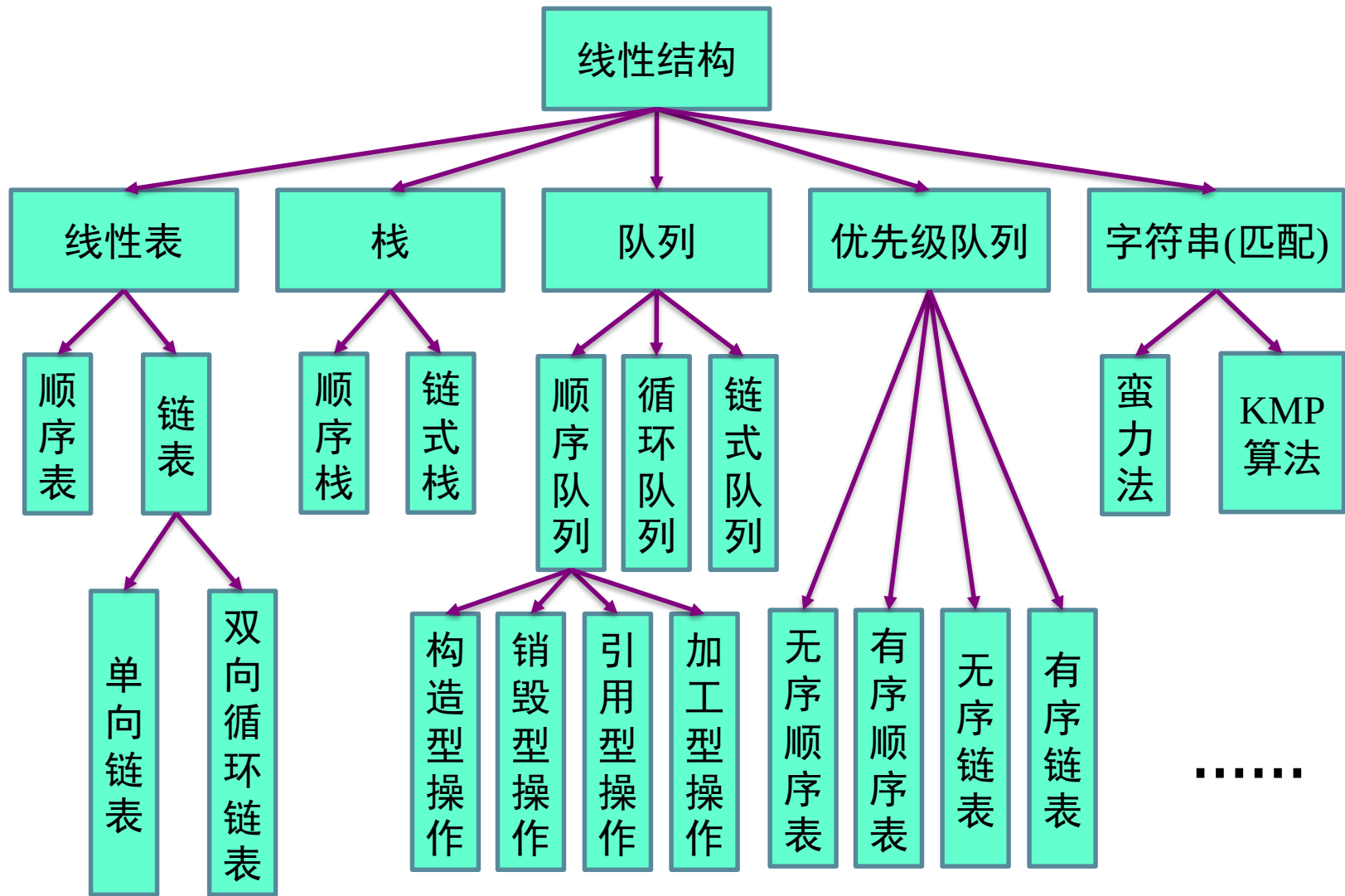
冯伟 (fengwei@tsinghua.edu.cn)

2024年10月15日





书接上文



.....

抽象数据类型：“分类” & “抽象”



How to Grow a Mind: Statistics, Structure, and Abstraction
Joshua B. Tenenbaum *et al.*
Science **331**, 1279 (2011);
DOI: 10.1126/science.1192788



... it can infer that a hierarchical organization for the novel objects better fits the similarities people see in these objects...



提纲

- 树的基本概念
- 二叉树的定义与性质
- 二叉树的表示、实现、遍历

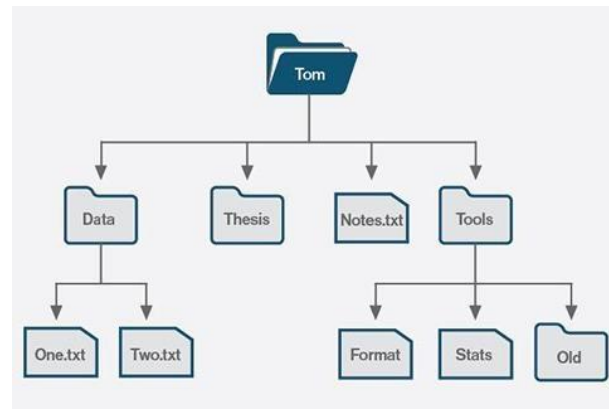
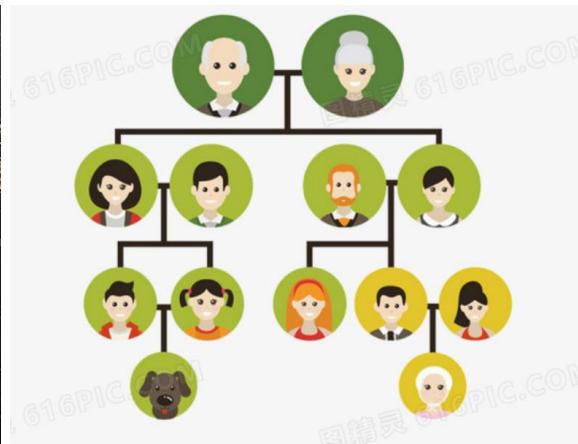


提纲

- 树的基本概念
- 二叉树的定义与性质
- 二叉树的表示、实现、遍历

树 (Tree)

树结构普遍存在于自然界、人类社会和信息系统中



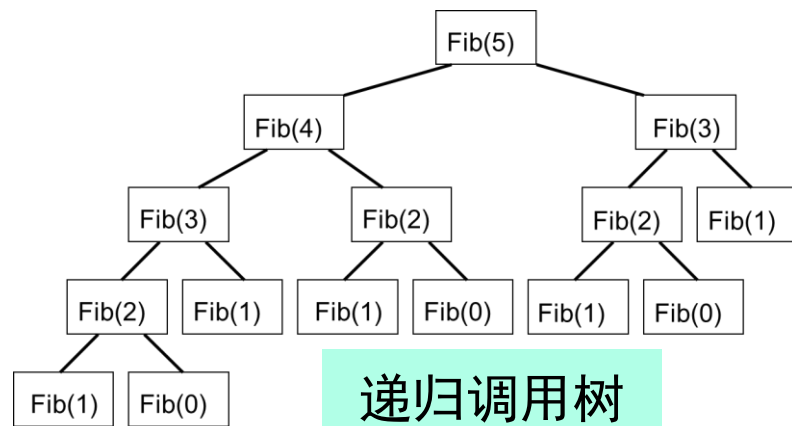
——图片来自网络



树

- 数据与算法中的树结构是对实际树结构的抽象，表现为以分支关系定义的层级结构
- 树是一类非常重要的非线性数据结构，可以
 - 存储和表示具有分支和层级关系的实际问题

```
long Fib( long n ) {  
    if ( n <= 1 ) return 1;  
    else  
        return Fib(n-1) + Fib(n-2);  
}
```

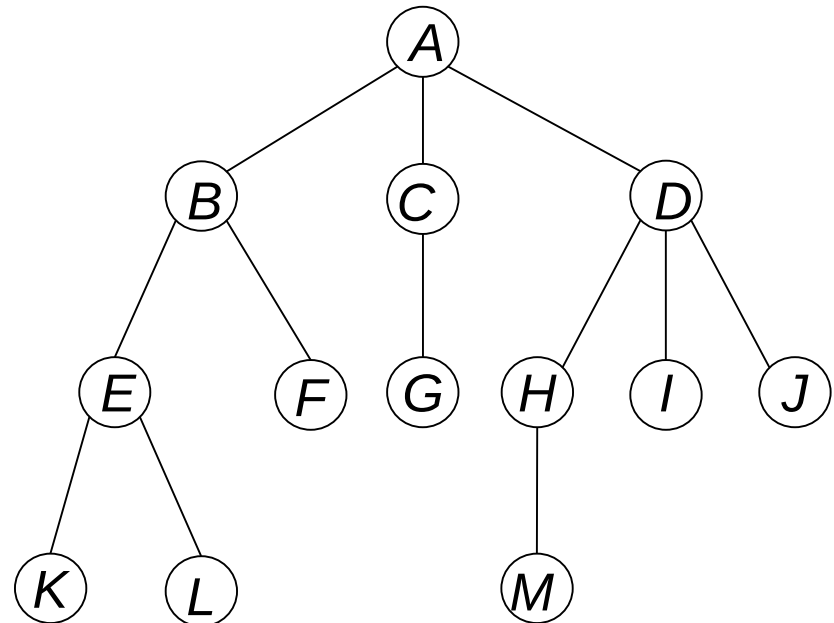


- 将线性表转化为树结构，大幅度提高查找效率（如二叉搜索树）
- 一些问题可通过构造树结构的方式求解（如霍夫曼编码）



树 - 定义

- 树是一种由一对多分支关系定义的层级结构
- 树是由 n ($n \geq 0$)个结点组成的有限集合（空树： $n = 0$ ），若 $n > 0$ ：
 - 有且仅有一个根(root)结点，它只有直接后继，无直接前驱
 - 除根以外的其它结点划分为 m ($m > 0$)个互不相交的有限集合， $T_0, T_1, \dots, T_{m-1}$ ，每个集合又是一棵树，并且称之为根的子树(sub-Tree)。每棵子树的根结点有且仅有一个直接前驱，但可以有零个或多个直接后继



根	$\text{root} = A$
子树	$T_1 = \{B, E, F, K, L\}$
子树	$T_2 = \{C, G\}$
子树	$T_3 = \{D, H, I, J, M\}$

树是一种递归的数据结构

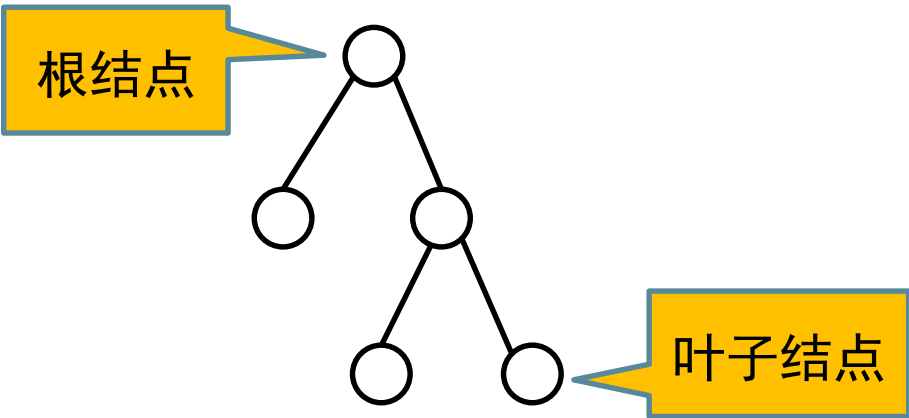
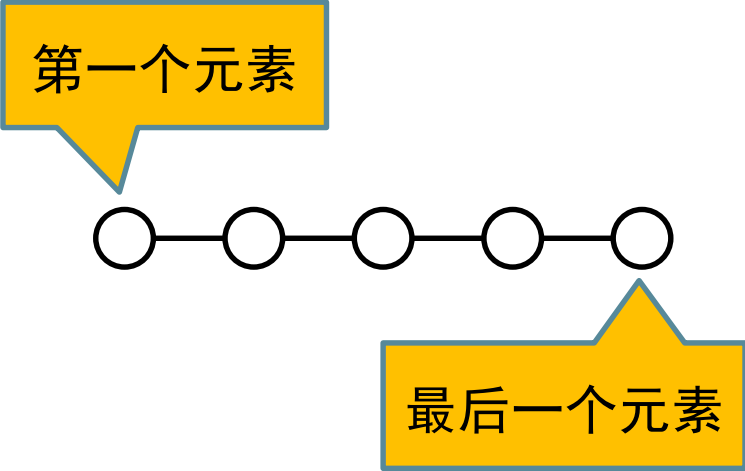


树 - 特点

- 树中的结点是分层次的
- 在一颗非空树中，有且仅有一个没有直接前驱的结点（根结点）
- 除根结点外，其余每个结点有且仅有唯一的直接前驱
- 树中所有结点可以有 ≥ 0 的直接后继
- 除根结点外，其余结点都有唯一的路径连接到根结点
 - 始于根结点，终止于该结点本身
 - 该路径相邻结点之间均为直接前驱与直接后驱的关系（线性结构）
 - 树中不存在封闭的回路（这是层次关系的一种体现）



树与线性结构比较

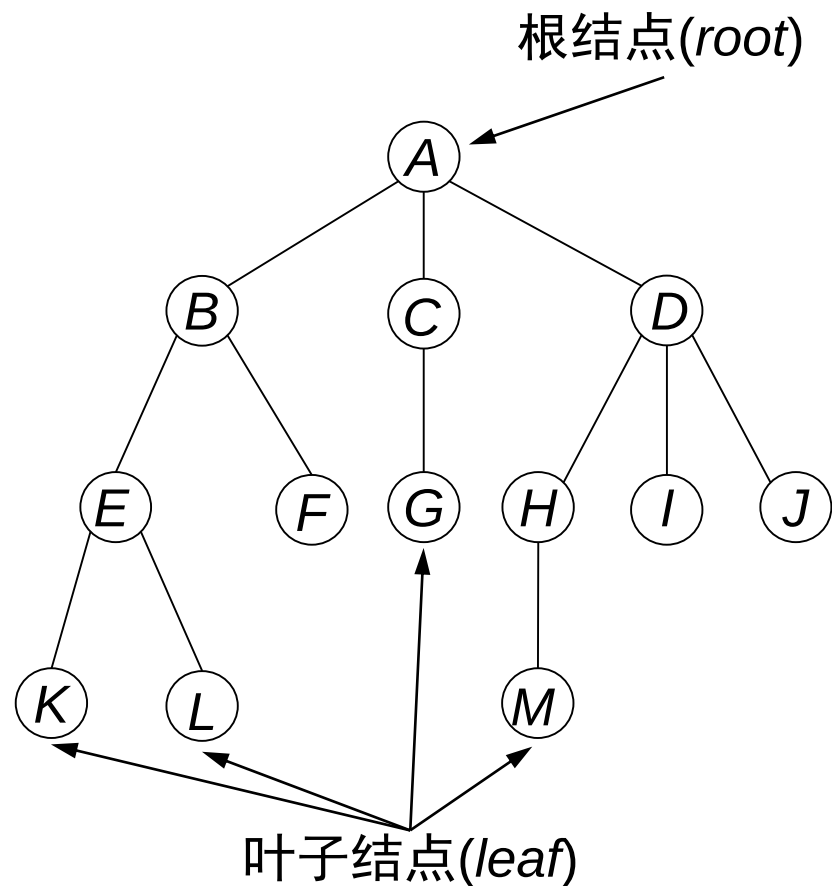


线性结构	树结构（非线性结构）
第一个数据元素（无前驱）	根结点（无前驱）
最后一个数据元素（无后继）	多个叶子结点（无后继）
其它元素：一个前驱、一个后继	其它结点：一个前驱、多个后继



树 - 基本术语

- **结点(node)**: 树的基本单位, 包括一个数据元素及指向其子树的分支
- **结点的度(degree)**: 结点的子树个数
 - ▣ **分支结点(branch)**: 度不为零
 - ▣ **叶子结点(leaf)**: 度为零
- **树的度**: 树中结点的度的最大值
- 如果树的度为 k , 我们称这棵树为 **k 叉树** (重点研究二叉树)



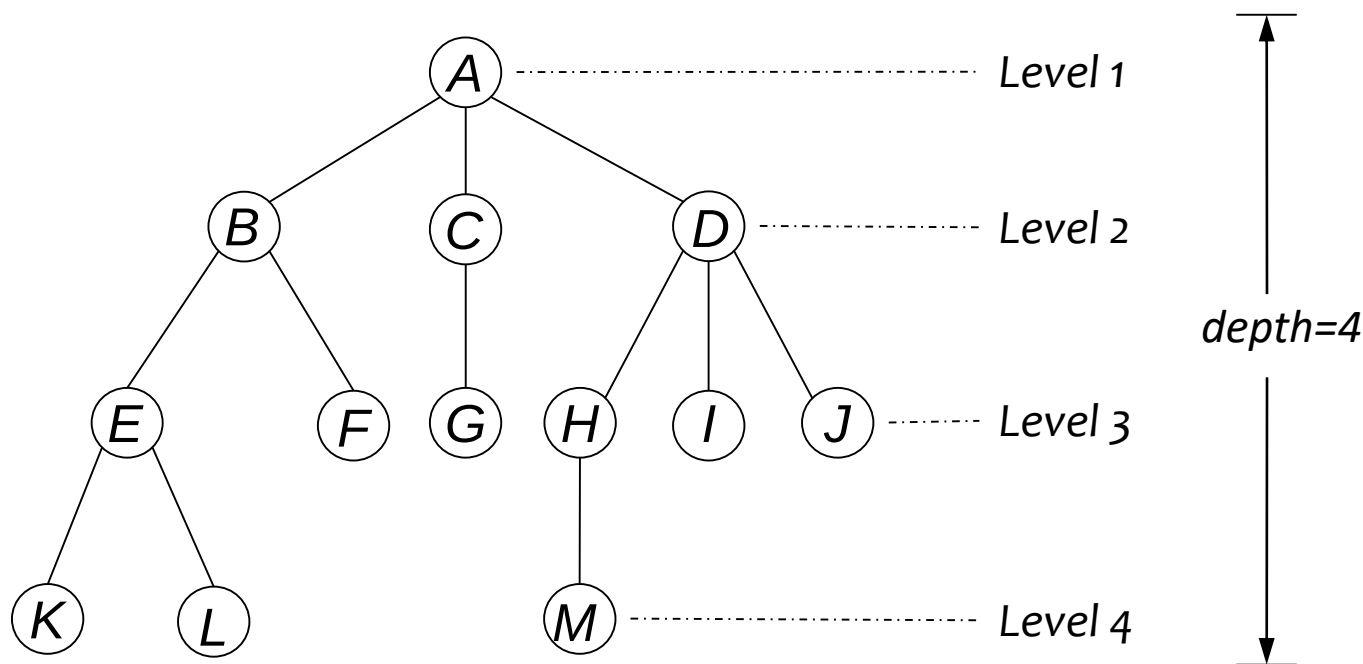


树 - 基本术语

- **孩子结点(child)**: 某个结点的直接后继, 或者说某个结点的子树的根
- 一个结点被称为其孩子结点的**双亲结点(parent)**, 或者**父结点**
- 具有相同双亲的结点互为**兄弟结点(sibling)**
- 某个结点的不同子树的根结点的孩子结点互为**堂兄弟结点(cousin)**
- 从根结点到某个结点的路径上的所有结点被称为这个结点的**祖先(ancestor)**
- 某个结点的所有子树中的结点称为这个结点的**子孙(descendant)**



树 - 基本术语



- **结点的层次**: 根结点的层次为1, 第 i 层的结点的子树根结点的层次为 $i + 1$
- **树的深度(depth)**: 树中结点 (叶子) 的最大层次, 也称树的高度(height)



树的性质

- 性质1：树中的结点数目等于所有结点的度数和加1
- 性质2：若 k 叉树的层次从1开始，第 i 层上至多 k^{i-1} 个结点($i \geq 1$)
- 性质3：深度为 h 的 k 叉树至多有 $\frac{k^h-1}{k-1}$ 个结点
- 性质4：有 n 个结点的 k 叉树最小深度为 $\lceil \log_k(n \times (k-1) + 1) \rceil$

(证明过程参考教材3.1.2.2)



提纲

- 树的基本概念
- 二叉树的定义与性质
- 二叉树的表示、实现、遍历



二叉树 (Binary Tree)

- 二叉树是 $k = 2$ 的 k 叉树
- 二叉树是一个包含 n 个结点的有限集，或者为空，或者满足：
 - 只有一个根结点（无直接前驱）
 - 除根结点以外的其他结点划分为 L 和 R 两个不相交的子集，每个集合又是一颗二叉树
 - L 称为根结点 A 的左子树， L 的根结点为根结点 A 的左孩子
 - R 称为根结点 A 的右子树， R 的根结点称为根结点 A 的右孩子

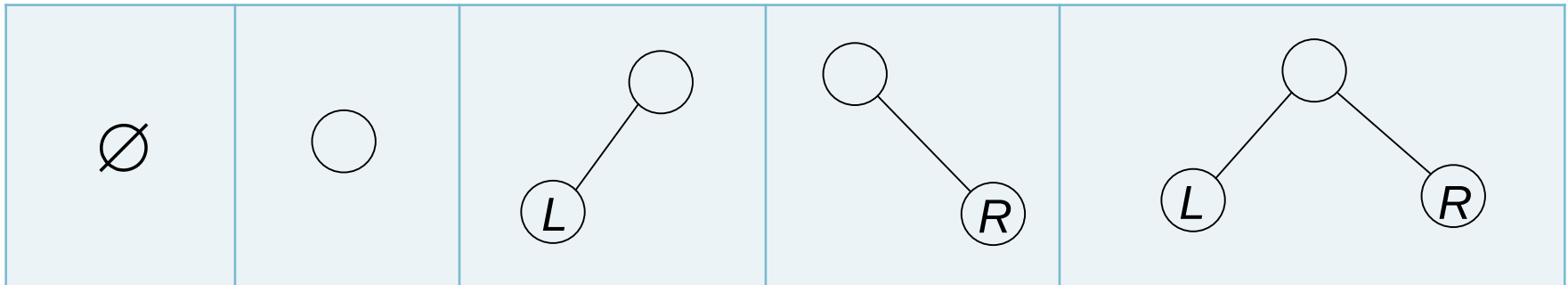
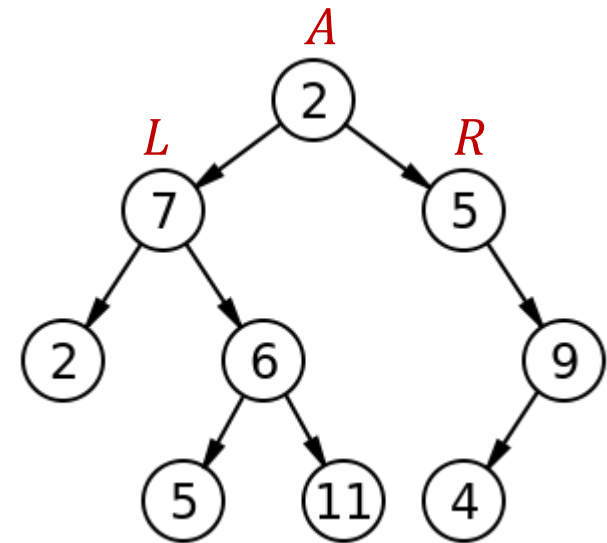
二叉树的例子



——图片来自2022.5.22宋同学朋友圈

二叉树 - 定义

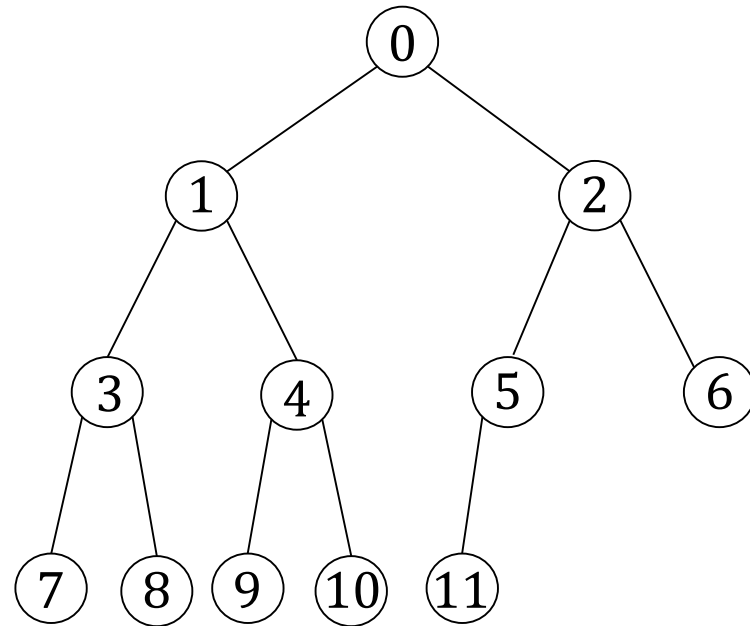
- 二叉树的子树有左右之分，不能互换
- 每个结点的度不大于2
- 只有三类结点：度为0，1，2
- 二叉树的五种结构形态如下：





二叉树的例子

- 根结点为0
- 左子树包含结点1,3,4,7,8,9,10
- 右子树包含结点2,5,6,11
- 结点1是左子树的根结点，也是结点0的左孩子结点
- 结点2是右子树的根结点，也是结点0的右孩子结点
- 以结点1为根的左子树中，包含以结点3为根的左子树和以结点4为根的右子树





二叉树的性质

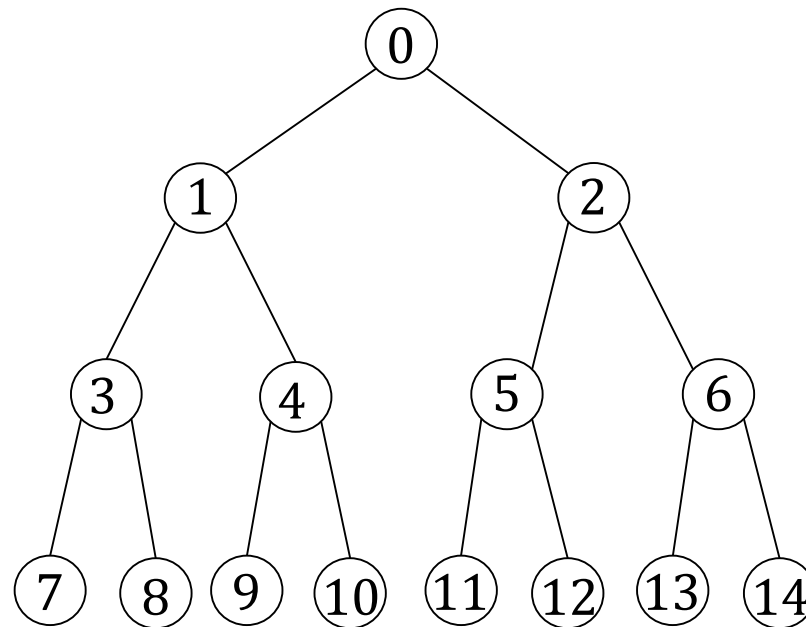
- 性质5：若二叉树有 n_0 个叶子结点， n_2 个2度结点，则 $n_0 = n_2 + 1$
- 性质6：若二叉树的层次从1开始，则第 i 层最多有 2^{i-1} 个结点 ($i \geq 1$) (性质2的特例)
- 性质7：深度为 h 的二叉树最多有 $2^h - 1$ 个结点 (性质3的特例)

(证明过程参考教材3.2.1.2)



满二叉树 (Full Binary Tree)

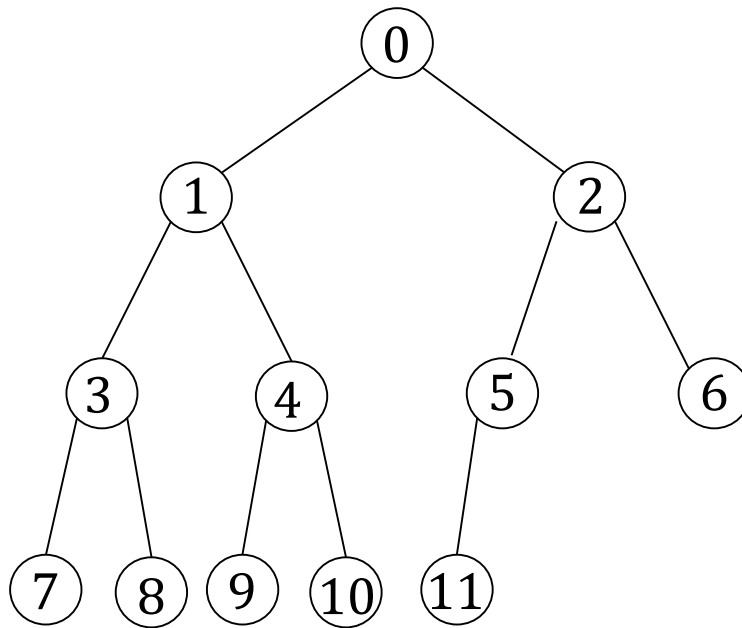
深度为 h ，且有 $2^h - 1$ 个结点的二叉树，称为满二叉树





完全二叉树 (Complete Binary Tree)

深度为 h 的二叉树， $1 \sim h - 1$ 层的结点数都达到最大值，第 h 层从右向左连续缺若干结点，称为完全二叉树



满二叉树是
完全二叉树的特例



二叉树的性质

- 性质8：具有 n 个结点的完全二叉树的深度为 $\lceil \log_2(n+1) \rceil$ （性质4的特例）[完全二叉树具有最小的深度]
 - 性质9：对 n 个结点的完全二叉树自顶向下、自左向右进行 0 至 $n-1$ 的编号，则对二叉树中任意一个编号为 i 的结点：
 - 若 $i = 0$ ，根结点，无双亲，否则结点 $\lfloor \frac{i-1}{2} \rfloor$ 为其双亲结点
 - 若 $2i+1 > n$ ，则该结点无左孩子；否则，结点 $2i+1$ 为其左孩子
 - 若 $2i+2 > n$ ，则该结点无右孩子；否则，结点 $2i+2$ 为其右孩子
 - 若 i 为非零偶数，则其左兄弟为 $i-1$ 结点
 - 若 i 为不大于 $n-2$ 的奇数，则其右兄弟结点为 $i+1$ ，否则无右兄弟
 - 结点 i 在 $\lceil \log_2(i+2) \rceil$ 层
- (证明过程参考教材3.2.1.2)



提纲

- 树的基本概念
- 二叉树的定义与性质
- 二叉树的表示、实现、遍历



二叉树的抽象数据类型：数据部分

ADT BinaryTree{

数据对象： $D = \{a_i | a_i \in ElemSet, i = 1 \dots n, n \geq 1\}$

数据关系：

若 $D = \emptyset$ ，则 $R = \emptyset$ ，称BinaryTree为空二叉树

若 $D \neq \emptyset$ ，则 $R = \{H\}$ ， H 是如下二元关系：

(1) 在 D 中存在唯一的称为根的数据元素 $root$ ，它在关系 H 下无前驱

(2) 若 $D - \{root\} \neq \emptyset$ ，则存在 $D - \{root\} = \{D_l, D_r\}$ ，且 $D_l \cap D_r = \emptyset$

(3) 若 $D_l \neq \emptyset$ ，则 D_l 中存在唯一的元素 x_l ， $\langle root, x_l \rangle \in H$ ，且存在 D_l 上的关系 $H_l \subset H$

若 $D_r \neq \emptyset$ ，则 D_r 中存在唯一的元素 x_r ， $\langle root, x_r \rangle \in H$ ，且存在 D_r 上的关系 $H_r \subset H$

$H = \{\langle root, x_l \rangle, \langle root, x_r \rangle, H_l, H_r\}$

(4) $(D_l, \{H_l\})$ 是一棵符合本定义的二叉树，称为根的左子树

$(D_r, \{H_r\})$ 是一棵符合本定义的二叉树，称为根的右子树



二叉树的抽象数据类型：基本操作

包括初始化，建立，遍历，查找，求深度，输出二叉树，判断树是否为空等

InitBiTree(&T)

- 操作结果：初始化空二叉树T

DestroyBiTree(&T)

- 初始条件：二叉树T已存在
- 操作结果：销毁二叉树T

CreateBiTree(&T,definition)

- 初始条件：definition给出二叉树T的定义
- 操作结果：按definition构造二叉树T

ClearBiTree(&T)

- 初始条件：二叉树T已存在
- 操作结果：将二叉树T清为空树

IsEmpty(&T)

- 初始条件：二叉树T已存在
- 操作结果：若T为空二叉树则返回TRUE，否则FALSE

GetDepth(T)

- 初始条件：二叉树T已存在
- 操作结果：返回二叉树T的深度



二叉树的抽象数据类型：基本操作

GetRoot(T)

- 初始条件：二叉树T已存在
- 操作结果：返回二叉树T的根

GetValue(T, e)

- 初始条件：二叉树T已存在，e是T中某个结点
- 操作结果：返回e的值

Assign(T, &e, value)

- 初始条件：二叉树T已存在，e是T中某个结点
- 操作结果：将结点e赋值为value

GetParent(T, e)

- 初始条件：二叉树T已存在，e是T中某个结点。
- 操作结果：若e是T的非根结点，返回它的双亲结点，否则返回空

LeftChild(T, e)

- 初始条件：二叉树T已存在，e是T中某个结点
- 操作结果：返回e的左孩子。若e无左孩子，则返回空

RightChild(T, e)

- 初始条件：二叉树T已存在，e是T中某个结点
- 操作结果：返回e的右孩子。若e无右孩子，则返回空



二叉树的抽象数据类型： 基本操作

InsertChild(T, p, LR, c)

- 初始条件：二叉树 T 已存在， p 指向 T 中某个结点， LR 为0或1，非空二叉树 c 与 T 不相交且右子树为空
- 操作结果：根据 LR 为0或1，插入 c 为 T 中 p 所指结点的左或右子树。 p 所指的原有左或右子树则成为 c 的右子树

DeleteChild(T, p, LR)

- 初始条件：二叉树 T 已存在， p 指向 T 中某个结点， LR 为0或1
- 操作结果：根据 LR 为0或1，删除 T 中 p 所指结点的左或右子树



二叉树的抽象数据类型： 基本操作

PreOrderTraverse(T, Visit())

- 初始条件：二叉树T已存在，Visit是对结点操作的函数。
- 操作结果：**前序遍历**T，对每个结点调用函数Visit一次且仅一次。一旦Visit()失败，则操作失败。

PostOrderTraverse(T, Visit())

- 初始条件：二叉树T已存在，Visit是对结点操作的函数
- 操作结果：**后序遍历**T，对每个结点调用函数Visit一次且仅一次；一旦Visit()失败，则操作失败

InOrderTraverse(T, Visit())

- 初始条件：二叉树T已存在，Visit是对结点操作函数。
- 操作结果：**中序遍历**T，对每个结点调用函数Visit一次且仅一次。一旦Visit()失败，则操作失败。

LevelOrderTraverse(T, Visit())

- 初始条件：二叉树T已存在，Visit是对结点操作的函数
- 操作结果：**层序遍历**T，对每个结点调用函数Visit一次且仅一次；一旦Visit()失败，则操作失败

}ADT BinaryTree

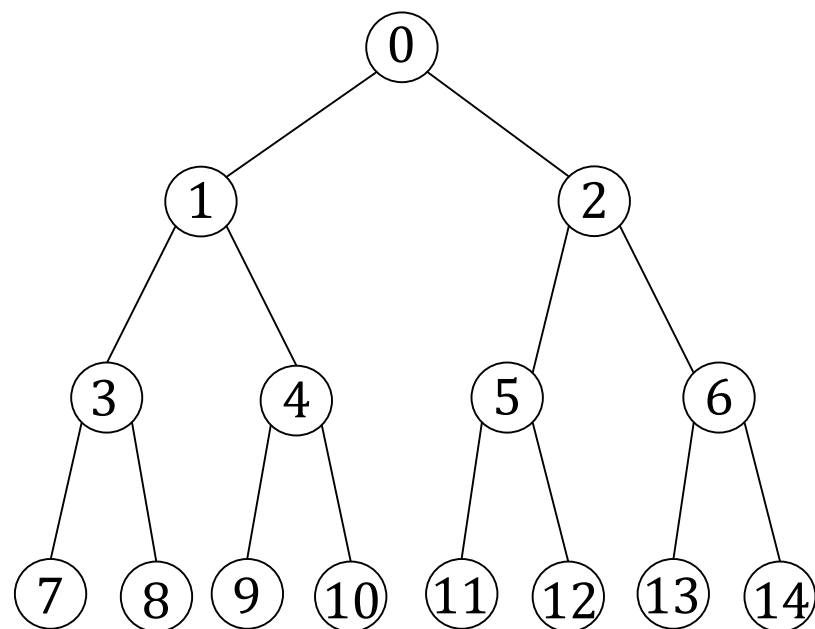


二叉树的顺序存储

将一个一般二叉树看成一个完全二叉树

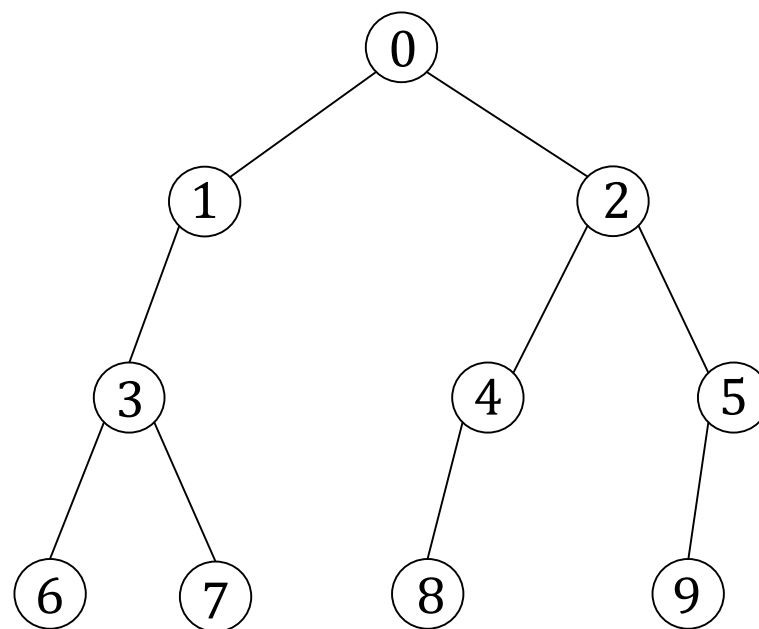
结点按层次从上到下，每层从左到右依次存入一维数组中

➤ 完全二叉树的顺序存储



0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
---	---	---	---	---	---	---	---	---	---	----	----	----	----	----

➤ 一般二叉树的顺序存储



0	1	2	3		4	5	6	7			8		9	
---	---	---	---	--	---	---	---	---	--	--	---	--	---	--



二叉树的顺序存储

- 将一个一般二叉树看成一个完全二叉树，结点按层次从上到下，每层从左到右依次存入一维数组中
- 可以直接确定每个结点的位置
- 对于二叉树的顺序表示， k 层二叉树需要 $2^k - 1$ 个存储单元
- 对于不完全二叉树，结点越少，空间效率越低
- 极端情况，对于只有 k 个结点的 k 层二叉树（单支树），仍然需要 $2^k - 1$ 个存储单元。空间效率为 $k/(2^k - 1)$ ， $k \geq 1$

顺序存储适合完全二叉树，对于一般二叉树，空间效率很低



二叉树的链式存储

➤ 二叉链表

- 数据域 (Data): 存储数据
- 指针域 (LChild and RChild): 存放左子树和右子树的指针



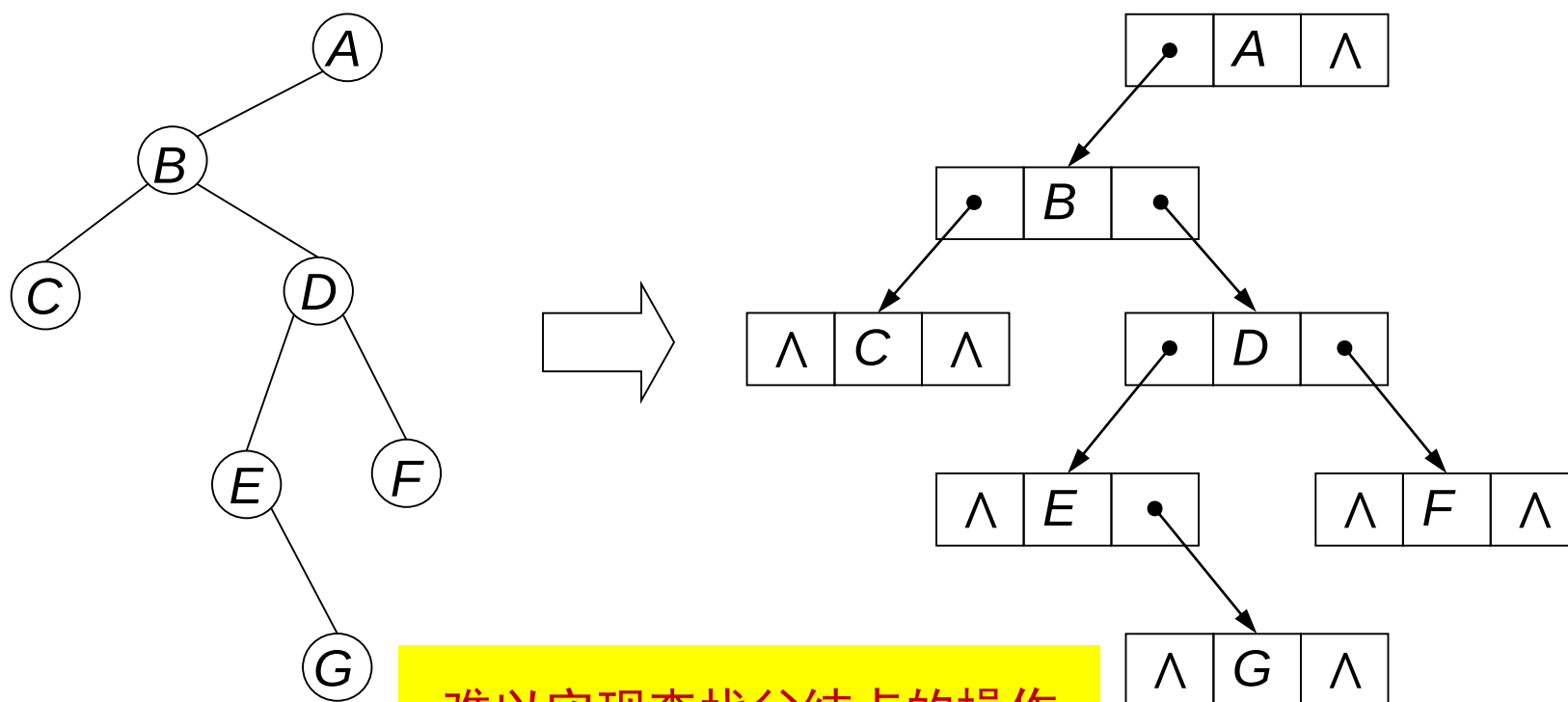
- 结点定义:

```
typedef struct tnode {  
    ElemType      data;  
    struct tnode  *lchild;  
    struct tnode  *rchild;  
} TreeNode;
```



二叉树的链式存储

二叉链表存储结构举例



难以实现查找父结点的操作



二叉树的链式存储

➤ 三叉链表

- 数据域 (Data): 存储数据
- 指针域 (Lchild, Rchild, Parent): 左子树、右子树、父亲结点的指针

LChild	Data	Parent	RChild
--------	------	--------	--------

- 结点定义:

```
typedef struct tnode {  
    ElemType      data;  
    struct tnode  *parent  
    struct tnode  *lchild;  
    struct tnode  *rchild;  
} TreeNode;
```



二叉树的遍历

- 二叉树遍历就是按照某种规律不重复的访问二叉树中所有结点
- 线性结构只有一个前驱、一个后继，遍历很容易；二叉树的遍历则是一个需要研究的问题
- 二叉树很多操作都需要基于遍历来实现
 - 二叉树的创建
 - 二叉树的清空和销毁
 - 求二叉树的高度



二叉树的遍历

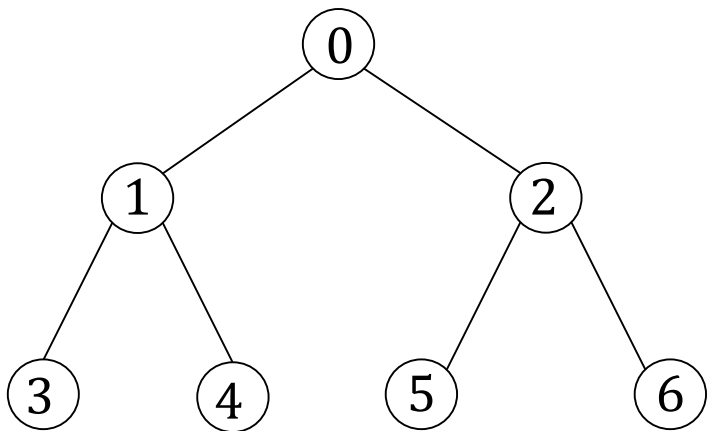
- 二叉树是一种递归结构，可以用递归思路来研究二叉树遍历方法
- 遍历过程对每个结点访问一次且仅访问一次，设访问根结点记作 V ，遍历根的左子树记作 L ，遍历根的右子树记作 R
- 根据访问结点的时机，遍历次序有三种：
 - 前序遍历 $V - L - R$
 - 中序遍历 $L - V - R$
 - 后序遍历 $L - R - V$
 - 还可以对二叉树自上而下逐层遍历，即层序遍历



二叉树 - 前序遍历

前序遍历 (Preorder Traversal)

1. 访问根结点
2. 遍历左子树
3. 遍历右子树



递归实现

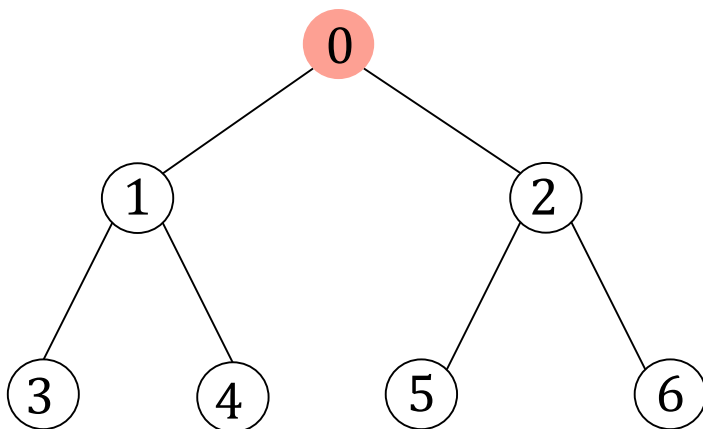
```
void BinaryTree::PreOrder (TNODE* t,  
    void(*Visit)(TNODE* ) ) {  
    if(t) {  
        Visit(t);  
        PreOrder(t->LChild,Visit);  
        PreOrder(t->RChild,Visit);  
    }  
}
```



二叉树 - 前序遍历

前序遍历 (Preorder Traversal)

1. 访问根结点
2. 遍历左子树
3. 遍历右子树



0

递归实现

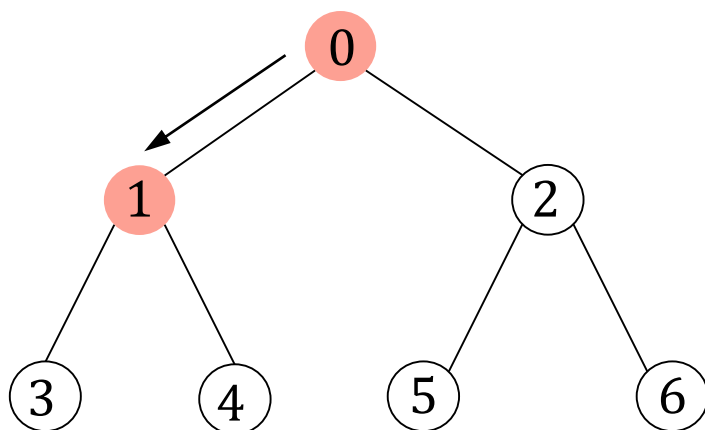
```
void BinaryTree::PreOrder (TNODE* t,  
    void(*Visit)(TNODE* ) ) {  
    if(t) {  
        Visit(t);  
        PreOrder(t->LChild,Visit);  
        PreOrder(t->RChild,Visit);  
    }  
}
```




二叉树 - 前序遍历

前序遍历 (Preorder Traversal)

1. 访问根结点
2. 遍历左子树
3. 遍历右子树



0-1

递归实现

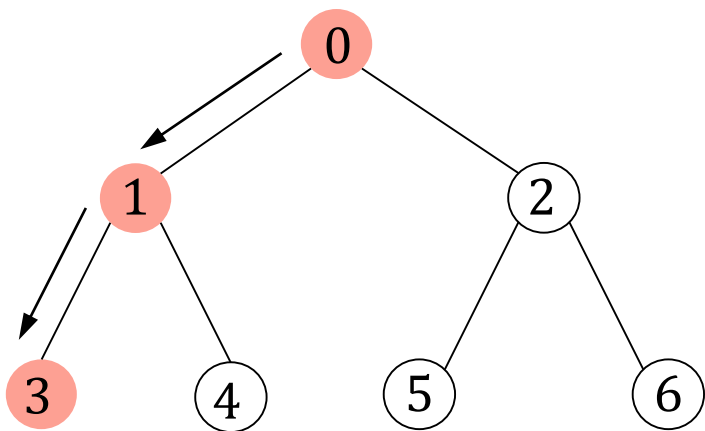
```
void BinaryTree::PreOrder (TNODE* t,  
    void(*Visit)(TNODE* ) ) {  
    if(t) {  
        Visit(t);  
        PreOrder(t->LChild,Visit);  
        PreOrder(t->RChild,Visit);  
    }  
}
```



二叉树 - 前序遍历

前序遍历 (Preorder Traversal)

1. 访问根结点
2. 遍历左子树
3. 遍历右子树



0-1-3

递归实现

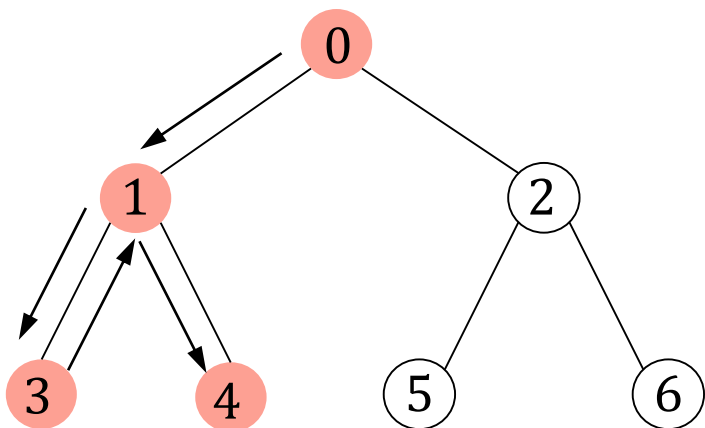
```
void BinaryTree::PreOrder (TNODE* t,  
    void(*Visit)(TNODE* ) ) {  
    if(t) {  
        Visit(t);  
        PreOrder(t->LChild,Visit);  
        PreOrder(t->RChild,Visit);  
    }  
}
```



二叉树 - 前序遍历

前序遍历 (Preorder Traversal)

1. 访问根结点
2. 遍历左子树
3. 遍历右子树



0-1-3-4

递归实现

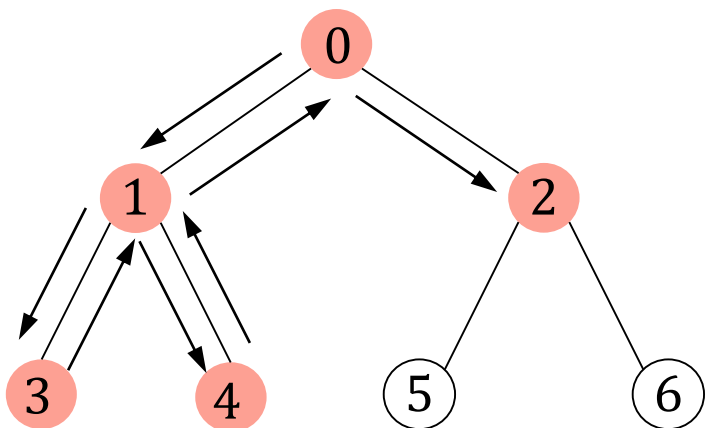
```
void BinaryTree::PreOrder (TNODE* t,  
    void(*Visit)(TNODE* ) ) {  
    if(t) {  
        Visit(t);  
        PreOrder(t->LChild,Visit);  
        PreOrder(t->RChild,Visit);  
    }  
}
```



二叉树 - 前序遍历

前序遍历 (Preorder Traversal)

1. 访问根结点
2. 遍历左子树
3. 遍历右子树



0-1-3-4-2

递归实现

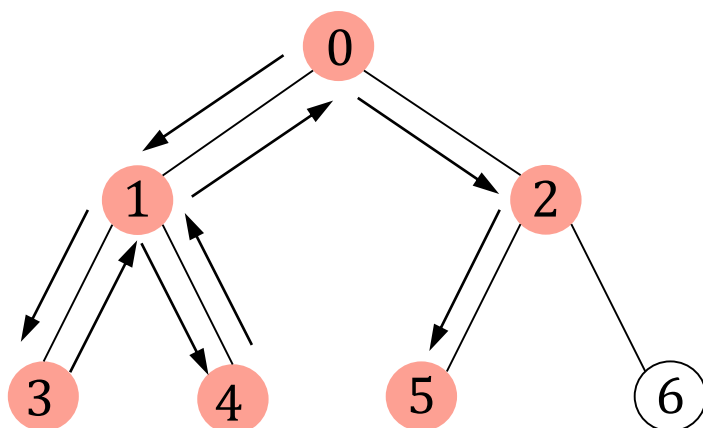
```
void BinaryTree::PreOrder (TNODE* t,  
    void(*Visit)(TNODE* ) ) {  
    if(t) {  
        Visit(t);  
        PreOrder(t->LChild,Visit);  
        PreOrder(t->RChild,Visit);  
    }  
}
```



二叉树 - 前序遍历

前序遍历 (Preorder Traversal)

1. 访问根结点
2. 遍历左子树
3. 遍历右子树



0-1-3-4-2-5

递归实现

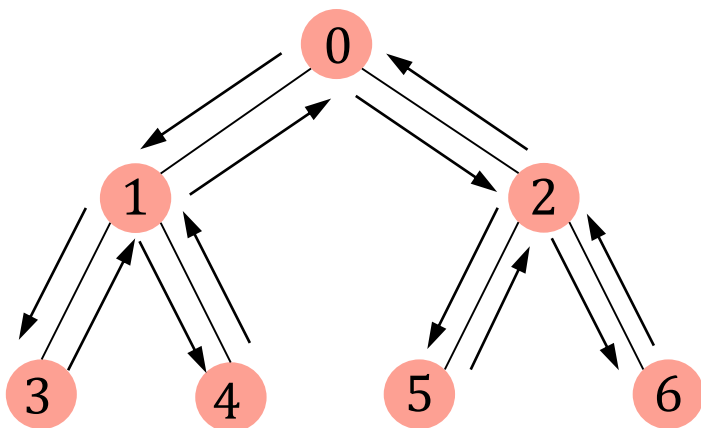
```
void BinaryTree::PreOrder (TNODE* t,  
    void(*Visit)(TNODE* ) ) {  
    if(t) {  
        Visit(t);  
        PreOrder(t->LChild,Visit);  
        PreOrder(t->RChild,Visit);  
    }  
}
```



二叉树 - 前序遍历

前序遍历 (Preorder Traversal)

1. 访问根结点
2. 遍历左子树
3. 遍历右子树



0-1-3-4-2-5-6

递归实现

```
void BinaryTree::PreOrder (TNODE* t,  
    void(*Visit)(TNODE* ) ) {  
    if(t) {  
        Visit(t);  
        PreOrder(t->LChild,Visit);  
        PreOrder(t->RChild,Visit);  
    }  
}
```

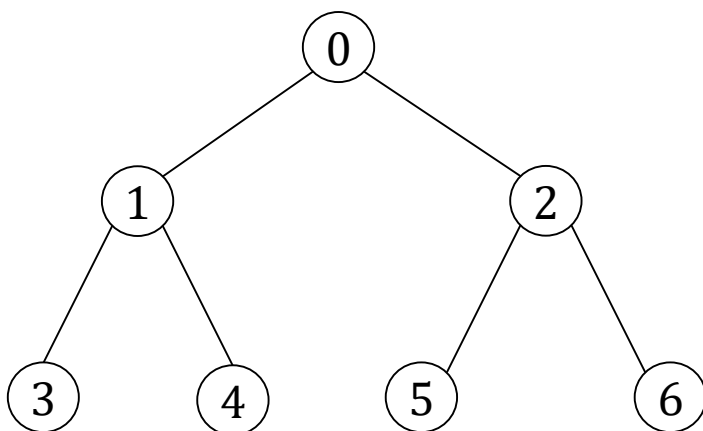


二叉树 - 中序遍历

1. 遍历左子树
2. 访问根结点
3. 遍历右子树

递归实现

```
void BinaryTree::InOrder ( TNode*t,  
    void(*Visit)(TNode* ) ) {  
    if(t) {  
        InOrder(t->lChild,Visit);  
        Visit(t);  
        InOrder(t->rChild,Visit);  
    }  
}
```



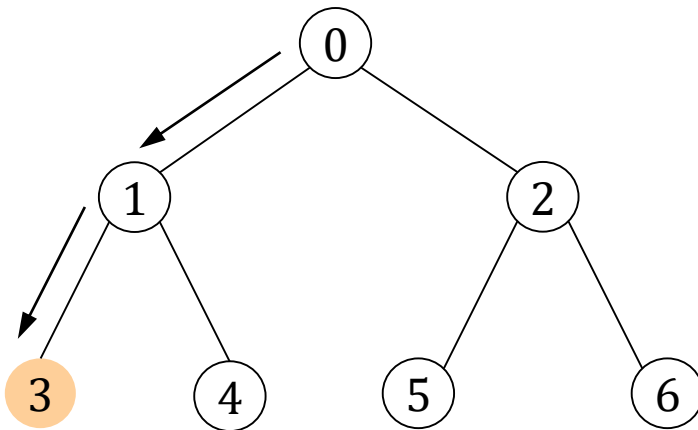


二叉树 - 中序遍历

1. 遍历左子树
2. 访问根结点
3. 遍历右子树

递归实现

```
void BinaryTree::InOrder ( TNode*t,  
    void(*Visit)(TNode* ) ) {  
    if(t) {  
        InOrder(t->lChild,Visit);  
        Visit(t);  
        InOrder(t->rChild,Visit);  
    }  
}
```



3

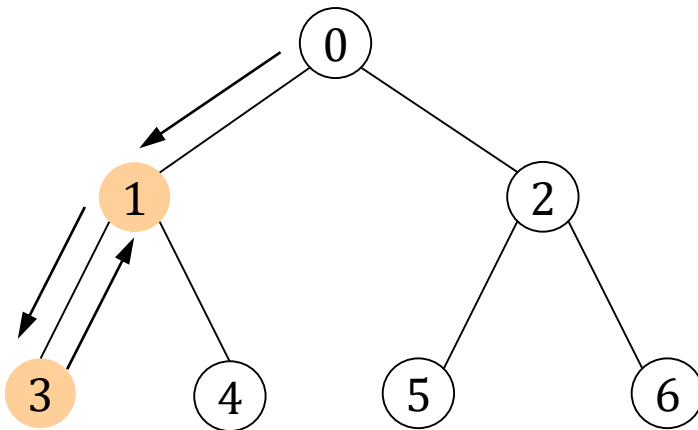


二叉树 - 中序遍历

1. 遍历左子树
2. 访问根结点
3. 遍历右子树

递归实现

```
void BinaryTree::InOrder ( TNode*t,  
    void(*Visit)(TNode* ) ) {  
    if(t) {  
        InOrder(t->lChild,Visit);  
        Visit(t);  
        InOrder(t->rChild,Visit);  
    }  
}
```



3-1

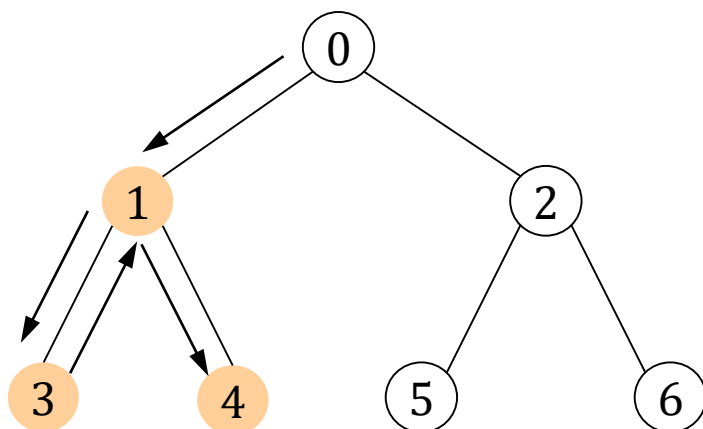


二叉树 - 中序遍历

1. 遍历左子树
2. 访问根结点
3. 遍历右子树

递归实现

```
void BinaryTree::InOrder ( TNode*t,  
    void(*Visit)(TNode* ) ) {  
    if(t) {  
        InOrder(t->lChild,Visit);  
        Visit(t);  
        InOrder(t->rChild,Visit);  
    }  
}
```



3-1-4

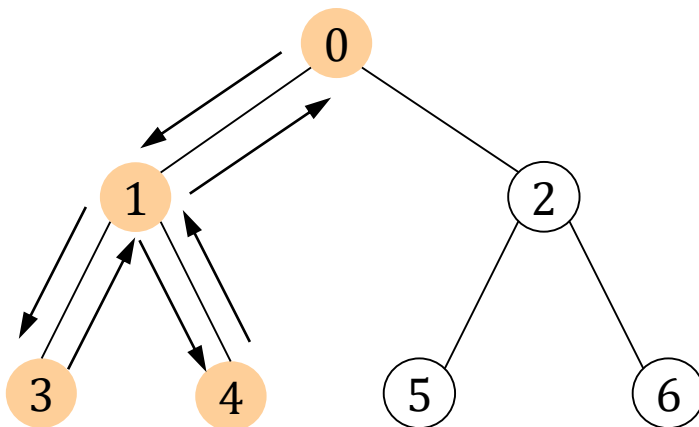


二叉树 - 中序遍历

1. 遍历左子树
2. 访问根结点
3. 遍历右子树

递归实现

```
void BinaryTree::InOrder ( TNode*t,  
    void(*Visit)(TNode* ) ) {  
    if(t) {  
        InOrder(t->lChild,Visit);  
        Visit(t);  
        InOrder(t->rChild,Visit);  
    }  
}
```



3-1-4-0

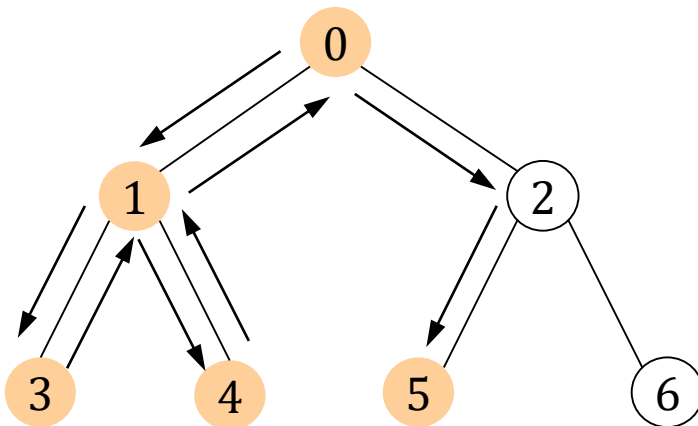


二叉树 - 中序遍历

1. 遍历左子树
2. 访问根结点
3. 遍历右子树

递归实现

```
void BinaryTree::InOrder ( TNode*t,  
    void(*Visit)(TNode* ) ) {  
    if(t) {  
        InOrder(t->lChild,Visit);  
        Visit(t);  
        InOrder(t->rChild,Visit);  
    }  
}
```



3-1-4-0-5

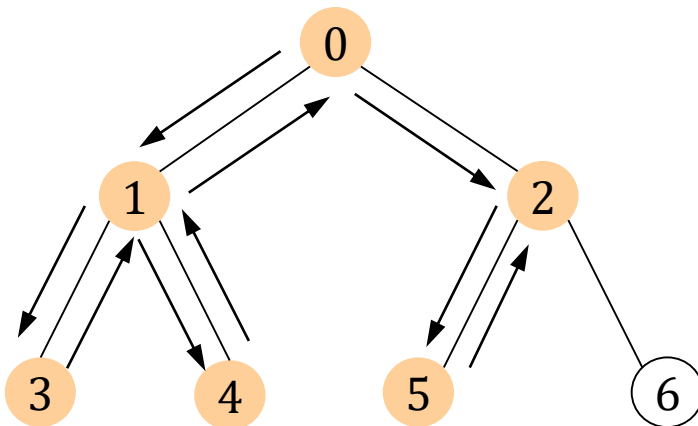


二叉树 - 中序遍历

1. 遍历左子树
2. 访问根结点
3. 遍历右子树

递归实现

```
void BinaryTree::InOrder ( TNode*t,  
    void(*Visit)(TNode* ) ) {  
    if(t) {  
        InOrder(t->lChild,Visit);  
        Visit(t);  
        InOrder(t->rChild,Visit);  
    }  
}
```



3-1-4-0-5-2

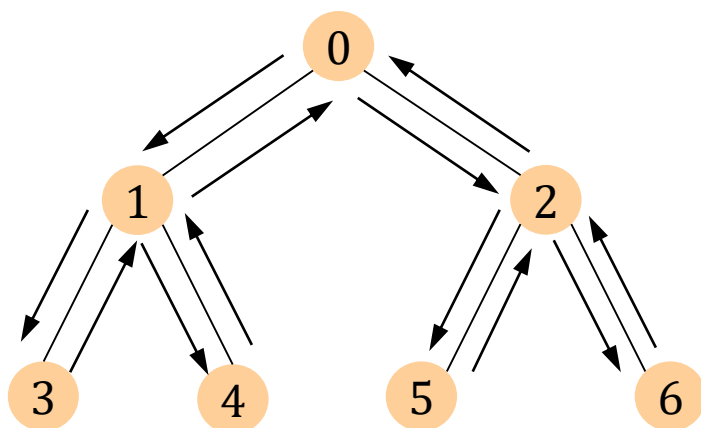


二叉树 - 中序遍历

1. 遍历左子树
2. 访问根结点
3. 遍历右子树

递归实现

```
void BinaryTree::InOrder ( TNode*t,  
    void(*Visit)(TNode* ) ) {  
    if(t) {  
        InOrder(t->lChild,Visit);  
        Visit(t);  
        InOrder(t->rChild,Visit);  
    }  
}
```



3-1-4-0-5-2-6

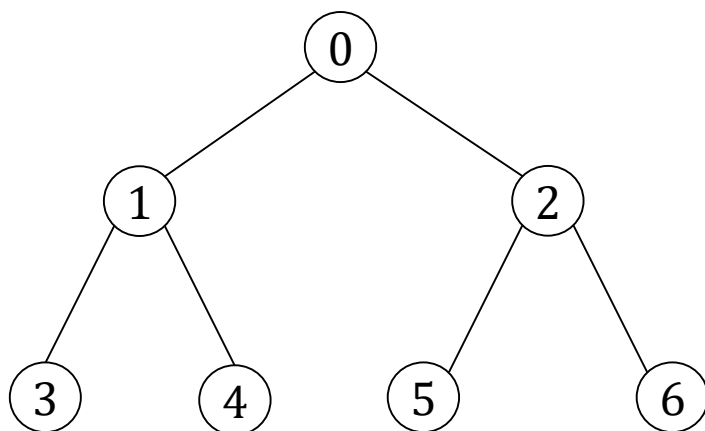


二叉树 - 后序遍历

1. 遍历左子树
2. 遍历右子树
3. 访问根结点

递归实现

```
void BinaryTree::PostOrder (TNODE *t,  
    void(*Visit)(TNODE* ) ) {  
    if(t) {  
        PostOrder(t->lChild,Visit);  
        PostOrder(t->rChild,Visit);  
        Visit(t);  
    }  
}
```



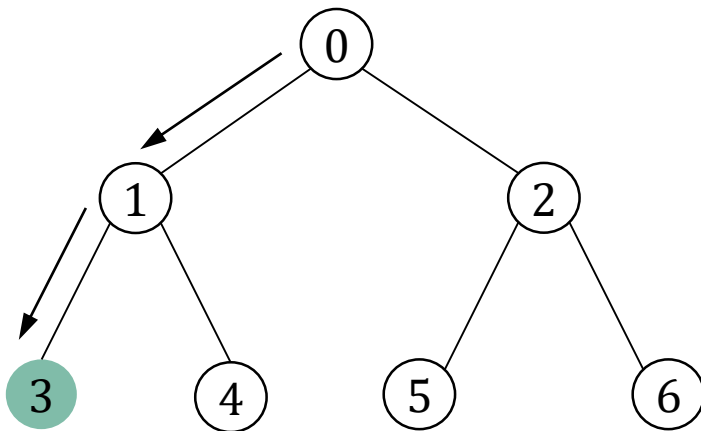


二叉树 - 后序遍历

1. 遍历左子树
2. 遍历右子树
3. 访问根结点

递归实现

```
void BinaryTree::PostOrder (TNODE *t,  
    void(*Visit)(TNODE* ) ) {  
    if(t) {  
        PostOrder(t->lChild,Visit);  
        PostOrder(t->rChild,Visit);  
        Visit(t);  
    }  
}
```



3

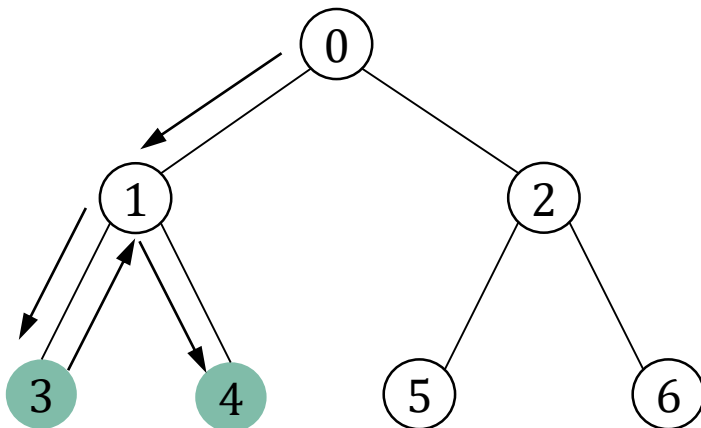


二叉树 - 后序遍历

1. 遍历左子树
2. 遍历右子树
3. 访问根结点

递归实现

```
void BinaryTree::PostOrder (TNODE *t,  
    void(*Visit)(TNODE* ) ) {  
    if(t) {  
        PostOrder(t->lChild,Visit);  
        PostOrder(t->rChild,Visit);  
        Visit(t);  
    }  
}
```



3-4

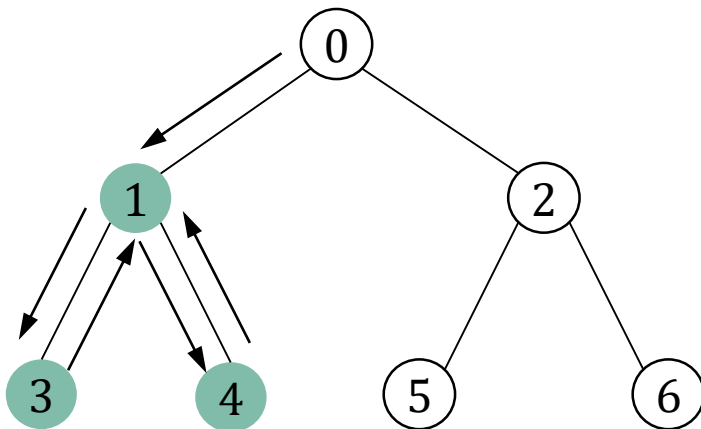


二叉树 - 后序遍历

1. 遍历左子树
2. 遍历右子树
3. 访问根结点

递归实现

```
void BinaryTree::PostOrder (TNODE *t,  
    void(*Visit)(TNODE* ) ) {  
    if(t) {  
        PostOrder(t->lChild,Visit);  
        PostOrder(t->rChild,Visit);  
        Visit(t);  
    }  
}
```



3-4-1

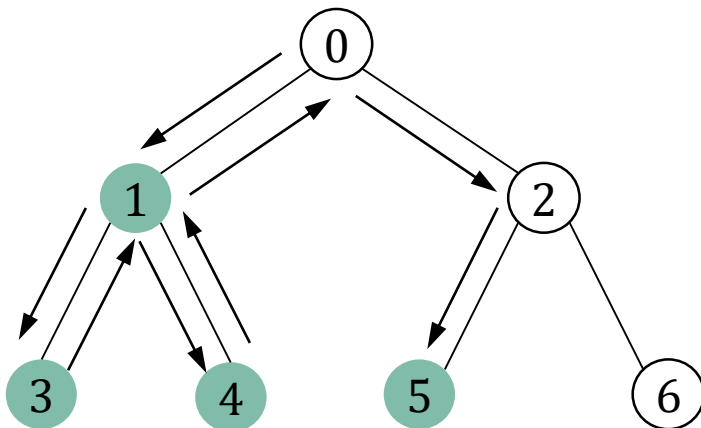


二叉树 - 后序遍历

1. 遍历左子树
2. 遍历右子树
3. 访问根结点

递归实现

```
void BinaryTree::PostOrder (TNODE *t,  
    void(*Visit)(TNODE* ) ) {  
    if(t) {  
        PostOrder(t->lChild,Visit);  
        PostOrder(t->rChild,Visit);  
        Visit(t);  
    }  
}
```



3-4-1-5

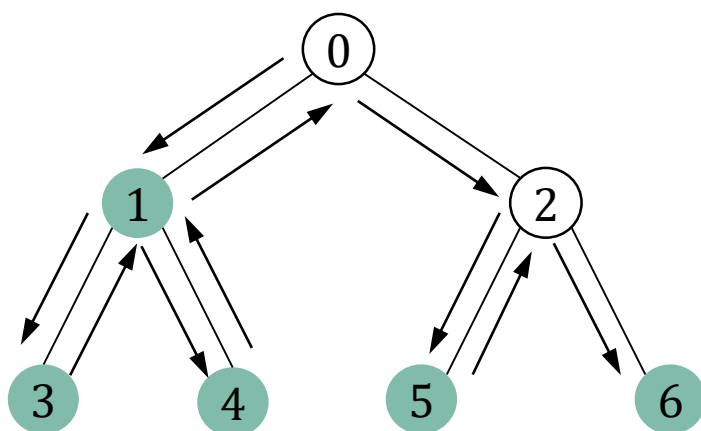


二叉树 - 后序遍历

1. 遍历左子树
2. 遍历右子树
3. 访问根结点

递归实现

```
void BinaryTree::PostOrder (TNODE *t,  
    void(*Visit)(TNODE* ) ) {  
    if(t) {  
        PostOrder(t->lChild,Visit);  
        PostOrder(t->rChild,Visit);  
        Visit(t);  
    }  
}
```



3-4-1-5-6

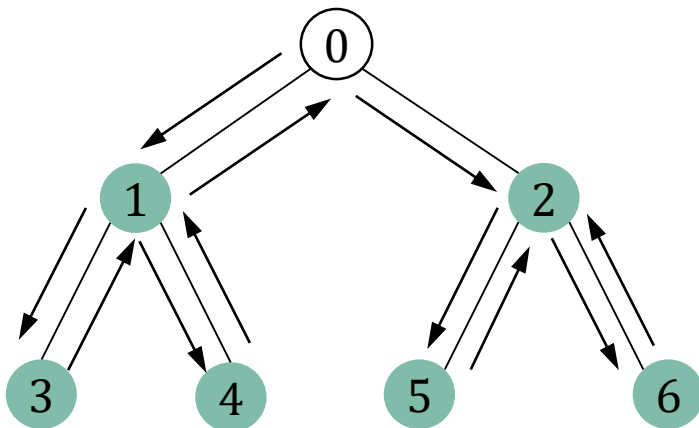


二叉树 - 后序遍历

1. 遍历左子树
2. 遍历右子树
3. 访问根结点

递归实现

```
void BinaryTree::PostOrder (TNODE *t,  
    void(*Visit)(TNODE* ) ) {  
    if(t) {  
        PostOrder(t->lChild,Visit);  
        PostOrder(t->rChild,Visit);  
        Visit(t);  
    }  
}
```



3-4-1-5-6-2

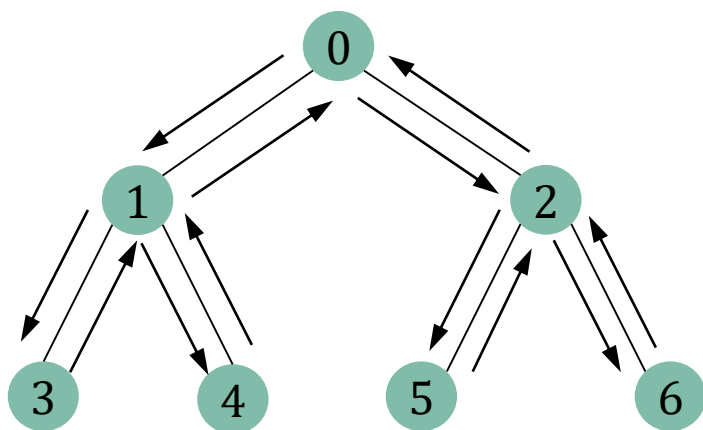


二叉树 - 后序遍历

1. 遍历左子树
2. 遍历右子树
3. 访问根结点

递归实现

```
void BinaryTree::PostOrder (TNODE *t,  
    void(*Visit)(TNODE* ) ) {  
    if(t) {  
        PostOrder(t->lChild,Visit);  
        PostOrder(t->rChild,Visit);  
        Visit(t);  
    }  
}
```



3-4-1-5-6-2-0



遍历的应用 - 运算表达式的树表示

运算表达式: $5 \times ((9 + 8) \times (4 \times 6)) + 7$

➤ 叶子结点存储数字, 其他结点存储运算符

➤ 括号的层次决定树的深度

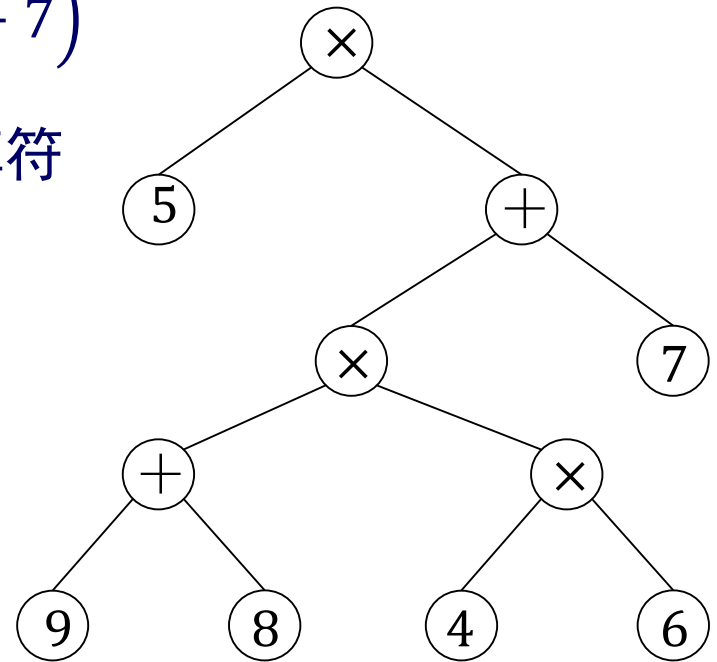
➤ **中序遍历**得到中缀表达式:

- $5 \times ((9 + 8) \times (4 \times 6)) + 7$

➤ **后序遍历**得到后缀表达式:

- $5 \ 9 \ 8 \ + \ 4 \ 6 \ \times \ \times \ 7 \ + \ \times$

- 进一步利用栈进行计算





遍历的应用 - 清空二叉树

- 清空二叉树，就是删除二叉树中的所有结点
- 可以先清空左、右子树，然后清空根结点

```
void BinaryTree::ClearBT(TNODE *t) {  
    if (t) {  
        ClearBT(t->LChild);  
        ClearBT(t->RChild);  
        delete t;  
        t = NULL;  
    }  
}
```

这是一个基于后序遍历的过程



遍历的应用 - 求二叉树的高度

```
int BinaryTree::BTDepth(TNODE *t) {  
    int depth;  
    if (t == NULL) return FALSE;           //若空，返回  
    if(t->LChild == NULL && t->RChild == NULL) //只有根结点  
        depth = 1;  
    else {  
        if(BTDepth(t->LChild) > BTDepth(t->RChild)) //求左右子树的高度  
            depth = 1 + BTDepth(t->lChild);  
        else  
            depth = 1 + BTDepth(t->rChild);           //取其中的大者加1  
    }  
    return depth; }  
}
```

这是一个基于后序遍历的过程



二叉树的非递归遍历

- 递归算法具有简洁、清晰的特点
- 有时需要消除递归以提高算法的时间/空间效率
- 分析递归算法的执行过程，就可以用非递归算法来实现它们
 - ❑ 建立一个栈，按照遍历规则从根结点出发对二叉树进行遍历
 - ❑ 并且逐层把未访问的结点或路径压入栈中
 - ❑ 当从各层返回时，退出栈顶结点或路径，访问栈顶结点或按其所指示的路径继续遍历
- 非递归用一个显示的栈代替递归工作栈，遍历过程与递归一致



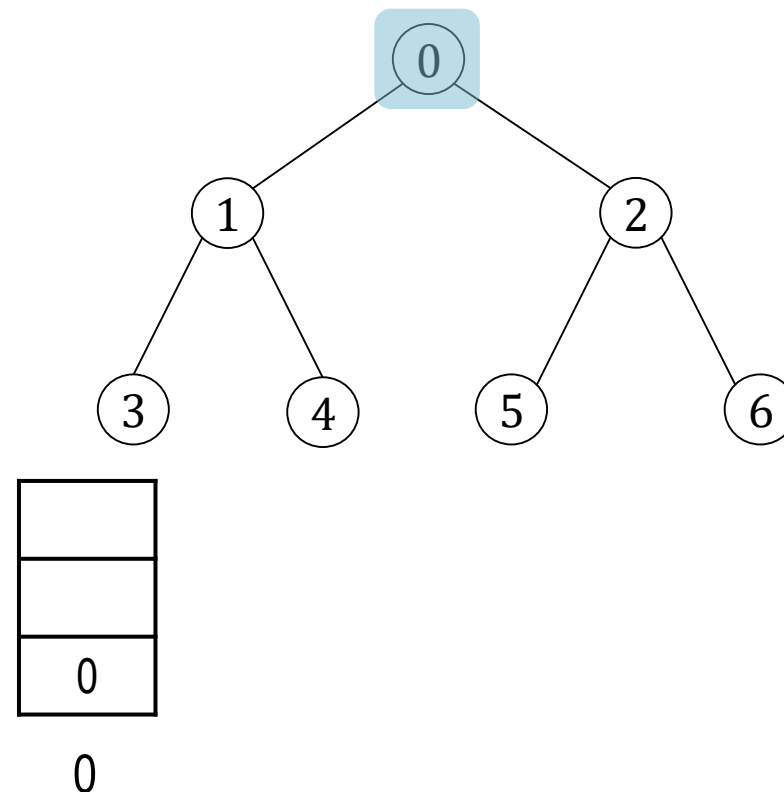
前序遍历的非递归实现

```
void BinaryTree :: PreOrderBT(TNODE* t, void(*Visit)(TNODE * )){  
    InitStack(S, 20);           //初始化元素为结点指针类型的栈S  
    TNODE *p = t;               //p指向当前结点，从根结点出发  
    while (p != NULL || !EmptyStack(S)) {  
        if (p != NULL) {        //若p不是空结点  
            Visit(p);            //访问结点p  
            Push(S, p);          //结点p入栈  
            p = p->LChild;       //使p的左孩子成为当前结点  
        }  
        else {  
            p = Pop(S);          //弹出栈顶结点并赋给p  
            p = p->RChild;       //使p的右孩子成为当前结点  
        }  
    }  
}
```



非递归前序遍历

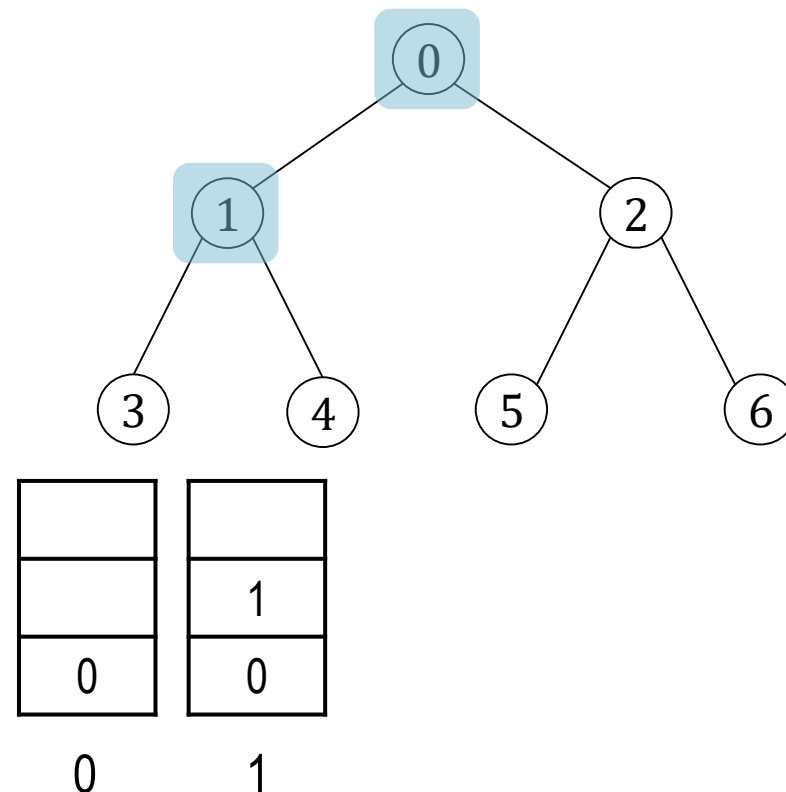
- 访问根结点0，结点0入栈，访问指针指向左孩子结点1；





非递归前序遍历

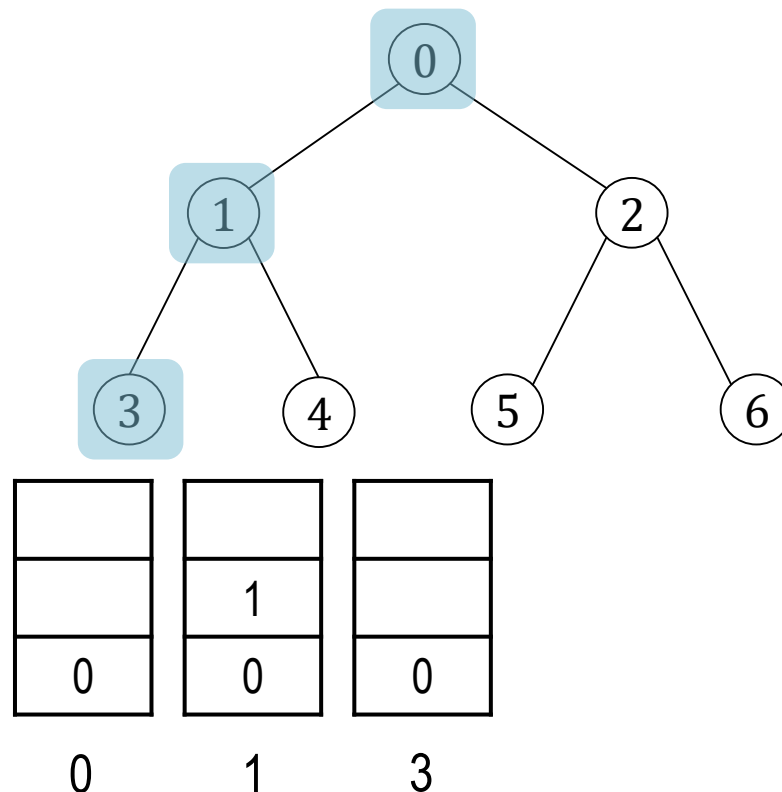
- 访问根结点0，结点0入栈，访问指针指向左孩子结点1；
- 访问结点1，结点1入栈；访问指针指向左孩子结点3；





非递归前序遍历

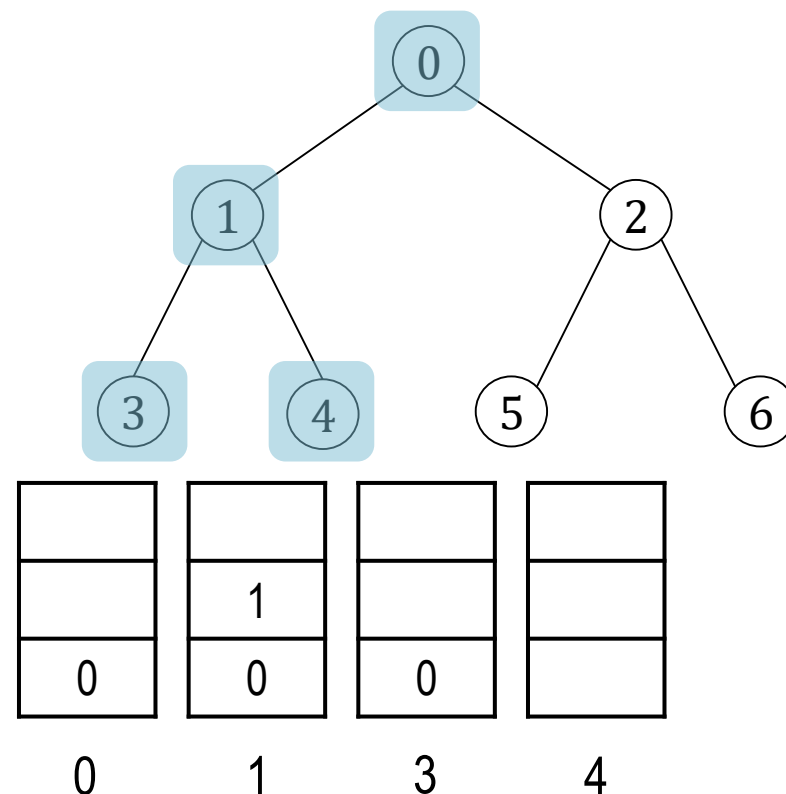
- 访问根结点0，结点0入栈，访问指针指向左孩子结点1；
- 访问结点1，结点1入栈；访问指针指向左孩子结点3；
- 访问结点3，为叶子结点，结点1出栈，访问指针指向结点4；





非递归前序遍历

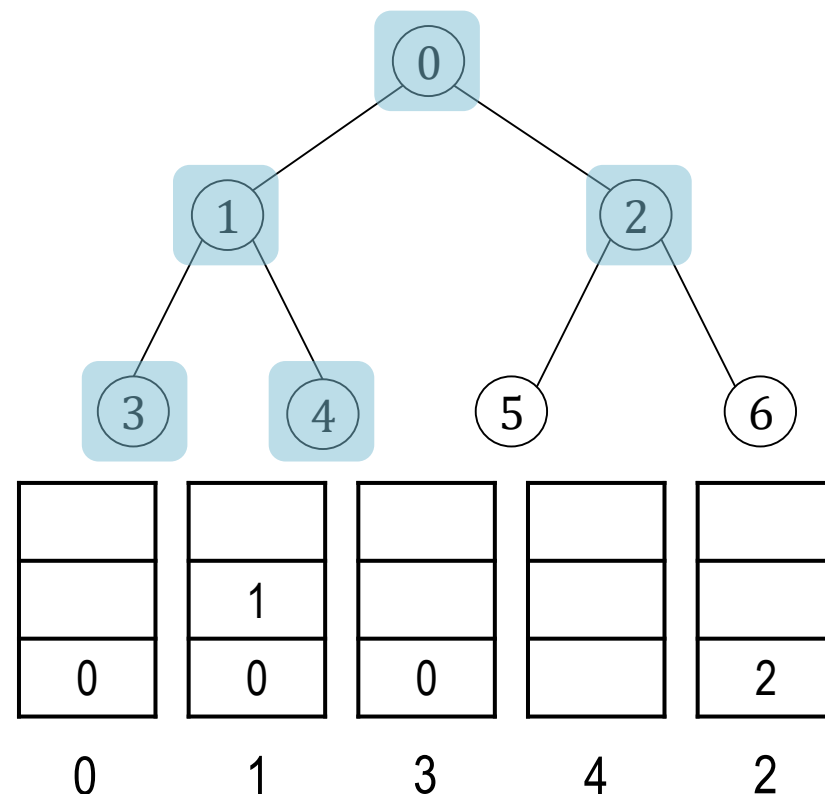
- 访问根结点0，结点0入栈，访问指针指向左孩子结点1；
- 访问结点1，结点1入栈；访问指针指向左孩子结点3；
- 访问结点3，为叶子结点，结点1出栈，访问指针指向结点4；
- 访问结点4，为叶子结点，结点0出栈，访问指针指向结点2；





非递归前序遍历

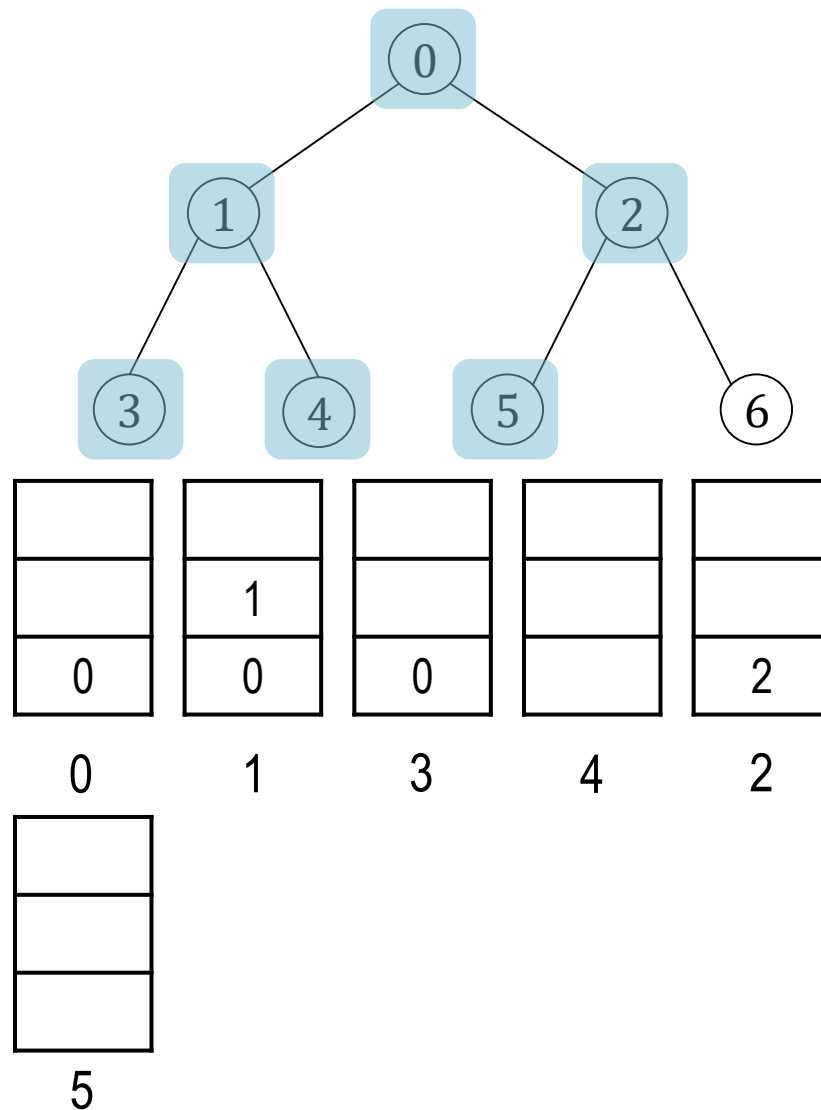
- 访问根结点0，结点0入栈，访问指针指向左孩子结点1；
- 访问结点1，结点1入栈；访问指针指向左孩子结点3；
- 访问结点3，为叶子结点，结点1出栈，访问指针指向结点4；
- 访问结点4，为叶子结点，结点0出栈，访问指针指向结点2；
- 访问结点2，结点2入栈，访问指针指向左孩子结点5；





非递归前序遍历

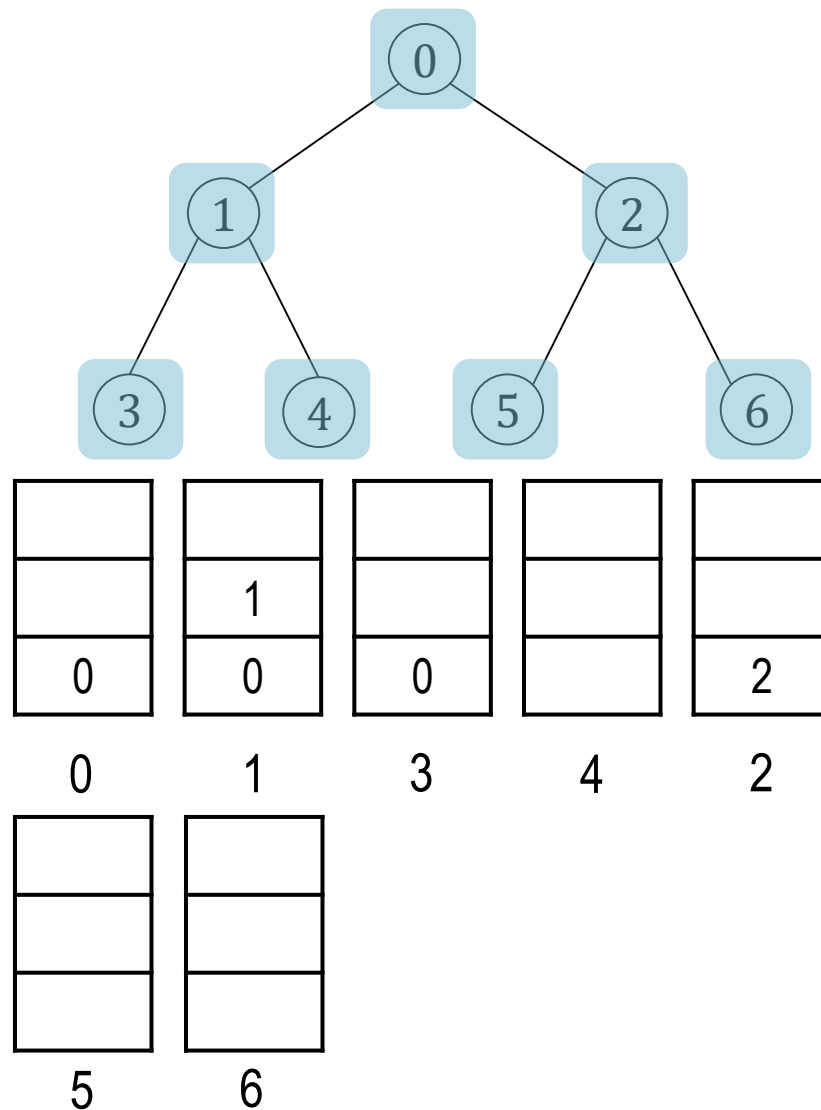
- 访问根结点0，结点0入栈，访问指针指向左孩子结点1；
- 访问结点1，结点1入栈；访问指针指向左孩子结点3；
- 访问结点3，为叶子结点，结点1出栈，访问指针指向结点4；
- 访问结点4，为叶子结点，结点0出栈，访问指针指向结点2；
- 访问结点2，结点2入栈，访问指针指向左孩子结点5；
- 访问结点5，为叶子结点，结点2出栈，访问指针指向结点6；





非递归前序遍历

- 访问根结点0，结点0入栈，访问指针指向左孩子结点1；
- 访问结点1，结点1入栈；访问指针指向左孩子结点3；
- 访问结点3，为叶子结点，结点1出栈，访问指针指向结点4；
- 访问结点4，为叶子结点，结点0出栈，访问指针指向结点2；
- 访问结点2，结点2入栈，访问指针指向左孩子结点5；
- 访问结点5，为叶子结点，结点2出栈，访问指针指向结点6；
- 访问叶子结点6，栈为空，遍历结束





二叉树的层序遍历

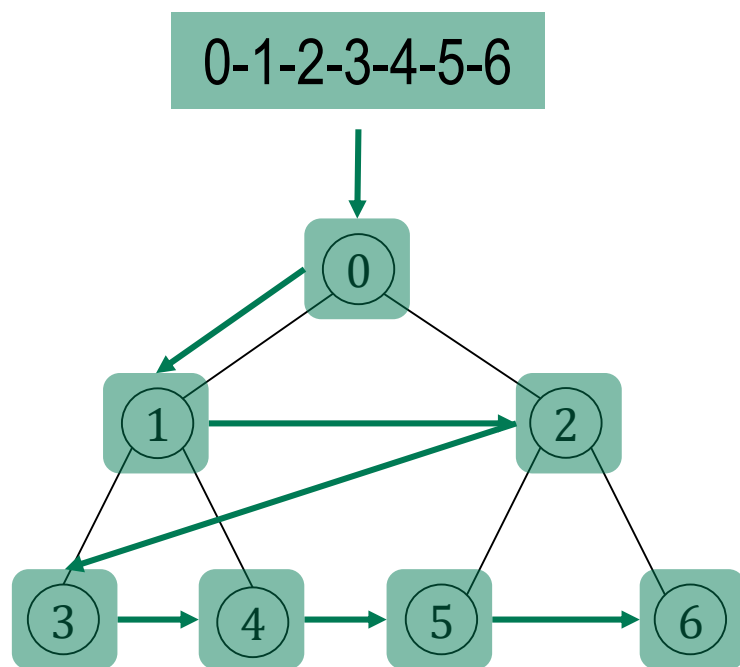
利用二叉树的层次特性，按照结点的层次从低到高逐层遍历

- 同一层次从左至右遍历
- 不具备递归性质
- 可以利用队列（先进先出）来实现

1. 根结点进队
2. 队头结点出队，进行访问操作，其左右孩子结点进队
3. 重复此步骤，直至队列为空，遍历完成



二叉树的层序遍历



0			
---	--	--	--

0入队

1	2		
---	---	--	--

0出队, 1,2入队

2	3	4	
---	---	---	--

1出队, 3,4入队

3	4	5	6
---	---	---	---

2出队, 5,6入队

4	5	6	
---	---	---	--

3出队

5	6		
---	---	--	--

4出队

6			
---	--	--	--

5出队

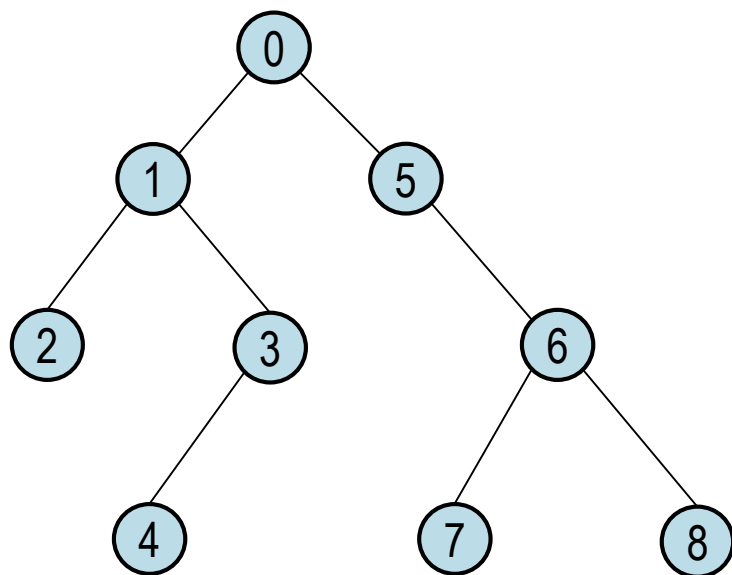
--	--	--	--

6出队, 队列空, 遍历完成

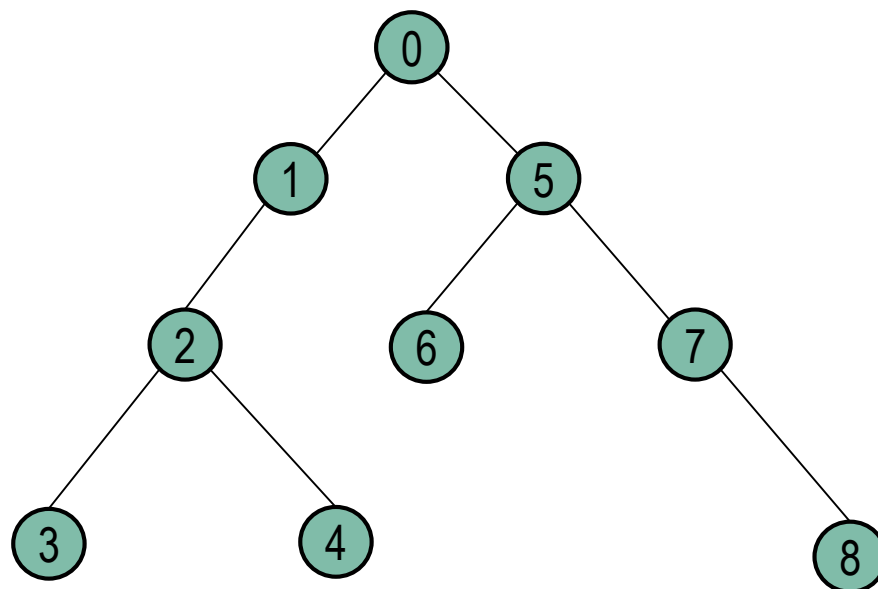


二叉树的建立

- 已知二叉树，通过遍历可以得到其前序序列、中序序列和后序序列
- 如果已知二叉树的某一前序序列，是否能确定这棵二叉树？



0-1-2-3-4-5-6-7-8



0-1-2-3-4-5-6-7-8

同一个前序序列可能对应不同二叉树，因此仅给定前序序列无法唯一确定



二叉树的建立

同时给定前序序列和中序序列可唯一地确定一棵二叉树

- 前序序列第一个元素给定了根结点
- 根据中序遍历规则，结合根结点，可以确定左右子树
- 返回前序序列，进而确定左右子树的根结点
- 返回中序序列，再确定左右子树的左右子树
-

前序遍历

0 - 1 - 2 - 3 - 4 - 5 - 6 - 7 - 8

中序遍历

2 - 1 - 4 - 3 - 0 - 5 - 7 - 6 - 8



二叉树的建立

0

前序
序列

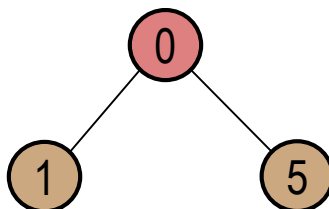
0-1-2-3-4-5-6-7-8

中序
序列

2-1-4-3-0-5-7-6-8



二叉树的建立



前序
序列

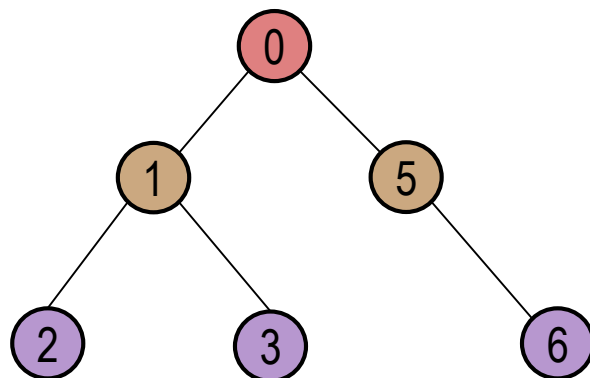
0-1-2-3-4-5-6-7-8
0 1-2-3-4 5-6-7-8

中序
序列

2-1-4-3-0-5-7-6-8
2-1-4-3 0 5-7-6-8



二叉树的建立



前序序列

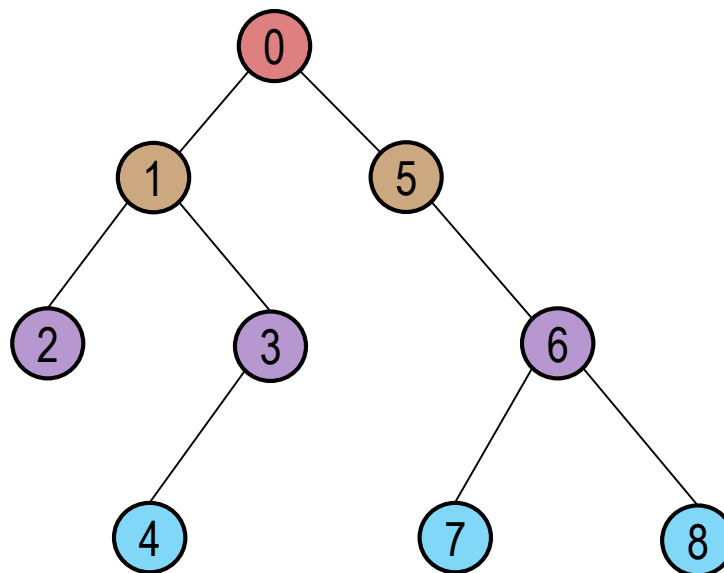
0-1-2-3-4-5-6-7-8
0 1-2-3-4 5-6-7-8
0 1 2 3-4 5 6-7-8

中序序列

2-1-4-3-0-5-7-6-8
2-1-4-3 0 5-7-6-8
2 1 4-3 0 5 7-6-8



二叉树的建立



前序序列

0-1-2-3-4-5-6-7-8
0 1-2-3-4 5-6-7-8
0 1 2 3-4 5 6-7-8
0 1 2 3 4 5 6 7 8

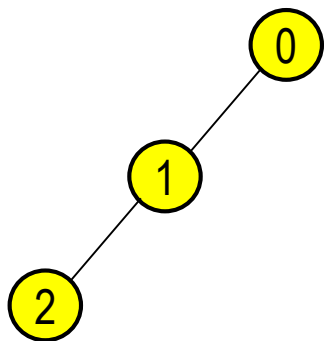
中序序列

2-1-4-3-0-5-7-6-8
2-1-4-3 0 5-7-6-8
2 1 4-3 0 5 7-6-8
2 1 4 3 0 5 7 6 8



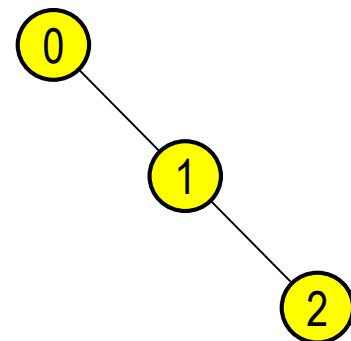
二叉树的建立

- 同一个前序序列可能对应不同二叉树，因此仅给定前序序列无法唯一确定
- 同时给定前序序列和中序序列可唯一地确定一棵二叉树
- 同时给定后序序列和中序序列能否唯一地确定一棵二叉树？ 
- 同时给定前序序列和后序序列能否唯一地确定一棵二叉树？ 



前序序列：0 1 2

后序序列：2 1 0





本讲小结

➤ 树的基本概念

- 定性概念、定量关系（几个主要性质）

➤ 二叉树的定义与性质

- 定性概念、定量关系（几个主要性质）

➤ 二叉树的表示、实现、遍历

- 前序遍历、中序遍历、后序遍历、层序遍历

- 基于遍历的其他操作

加油同学们!

